Master's Thesis

Vehicle Speed Estimation Based on Artificial Intelligence Algorithms

Author:
Andrzej Skrodzki

Budapest University of Technology and Economics
Department of Control for Transportation and Vehicle Systems

Supervisor:
Donát Rácz

Master's Thesis,
November 22, 2025

Contents

Introduction

Accurate vehicle speed estimation plays a critical role in modern automotive systems, contributing to various functionalities such as tire slip calculation, advanced driver-assistance systems (ADAS), intelligent transportation systems (ITS), and autonomous driving technologies. Conventionally, the measurement of vehicle speed has been accomplished through two primary methods: direct measurement via wheel speed sensors or indirect derivation using GPS data. However, it must be acknowledged that these methodologies are not without their limitations. The functionality of wheel speed sensors can be compromised by factors such as wheel slip or defects in the sensor itself. Concurrently, the reliability of GPS signals is often compromised in specific environments, including tunnels, urban canyons, and dense forests.

In this context, the Controller Area Network (CAN) bus, which facilitates communication between electronic control units (ECUs) in vehicles, offers a promising alternative source of information for estimating vehicle speed. The CAN bus transmits various sensor readings and control signals continuously, with many of these signals being correlated with vehicle dynamics. The utilization of these signals for speed estimation has the potential to enhance system robustness, particularly in scenarios where traditional methods are rendered ineffective.

The advent of artificial intelligence (AI) has led to a surge in the utilization of data-driven methodologies for addressing intricate estimation challenges. Artificial intelligence (AI) models, particularly those designed for time-series data, such as recurrent neural networks (RNNs), are well-suited to capture the temporal dependencies and nonlinear relationships inherent in CAN signals. When trained effectively, these models can provide real-time, accurate estimations of vehicle speed based solely on CAN data.

This thesis explores the development and validation of AI-based models for real-time vehicle speed estimation using CAN signal data from a passenger car. The estimation performance is evaluated against GPS measurements, which serve as ground truth. Moreover, the study explores hybrid approaches that integrate artificial intelligence (AI) with conventional filtering methods, with the objective of combining the strengths of both paradigms. The enhancement of the reliability and accuracy of speed estimation contributes to the development of safer and more intelligent vehicle systems.

Chapter 1

Vehicle speed

1.1 Defining Vehicle Speed

The speed of an arbitrary point in space is defined as the magnitude of the velocity vector of that point, that is considered in the inertial reference frame ($\mathcal{R}_{[x_0,y_0,z_0]}$, later denoted as $\mathcal{R}_0$ for simplicity). The velocity vector can be written as the time derivative of the position vector, where the position vector points from the origin of the inertial reference frame to the chosen moving point in space. However, when considering a moving vehicle in the same space, a relatively constrained set of points has to be considered instead of one moving point. In order to use the previously mentioned formula for determining the speed of the vehicle, one point has to be chosen on the vehicle that will represent the whole. This will be the center of gravity (CoG) that is the average location of the weight of the vehicle. With this in mind the vehicle speed can be determined as follows:

$$v_{CoG} = \|[\mathbf{v}_{CoG}]_{\mathcal{R}_0}\| = \left\| \frac{d[\mathbf{r}_{CoG}]_{\mathcal{R}_0}}{dt} \right\|, \tag{1.1}$$

where v_{CoG} is the vehicle speed, $[\mathbf{v}_{CoG}]_{\mathcal{R}_v}$ is the velocity vector of the CoG in the inertial reference frame and $[\mathbf{r}_{CoG}]_{\mathcal{R}_0}$ is the position vector of the CoG in the inertial reference frame. This way the velocity and the position vectors can be expressed in a 3 dimensional space as:

- position vector: $[\mathbf{r}_{CoG}]_{\mathcal{R}_0} = \begin{bmatrix} r_x \\ r_y \\ r_z \end{bmatrix}_{\mathcal{R}_0}$
- velocity vector: $[\mathbf{v}_{CoG}]_{\mathcal{R}_0} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}_{\mathcal{R}_0}$

As the moving vehicle in space is considered as a 3 dimensional body, the orientation of

it can be described with a body-fixed reference frame denoted by $\mathcal{R}_{[x_v,y_v,z_v]}$ (later denoted by $\mathcal{R}_v$ for simplicity), where x_v points forward along the vehicle frame's longitudinal axis, y_v points sideways to the left along the lateral axis and z_v points upwards to complete the right-handed coordinate system. Let this coordinate system be placed in the CoG of the vehicle. With the help of this body-fixed coordinate system the previously determined velocity vector ($[\mathbf{v}_{CoG}]_{\mathcal{R}_0}$) can be expressed in the body-fixed reference frame with the help of the rotational transformation matrix $T_{\mathcal{R}_0 \to \mathcal{R}_v}$ between the two reference frames. The rotational transformation matrix can be obtained from the rotational matrices around the three axes, namely x_0, y_0 and z_0. The angles of rotation around the axes are the following:

- angle of rotation around axis x_0: roll - ϕ
- angle of rotation around axis y_z: pitch - θ
- angle of rotation around axis z_0: yaw - ψ

From here the transformation matrix is as follows:

$$T_{\mathcal{R}_0 \to \mathcal{R}_v} = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi) \tag{1.2}$$

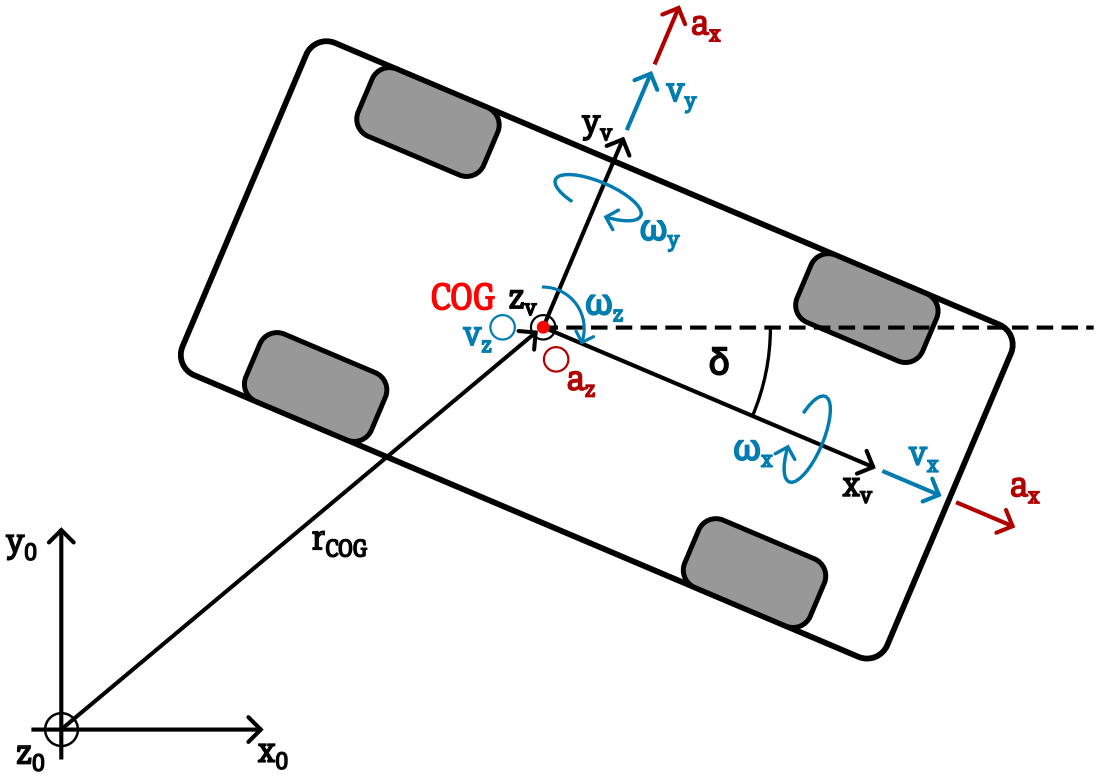


Figure 1.1: Vehicle speeds

1.2 Measurements for determining vehicle speed

There are various ways to determine the speed of the vehicle. In this section the focus is on exploring these different ways and analyzing them.

1.2.1 Engine speed

The first way is calculating the wheel and vehicle speed from the rotational speed of the engine [**engine_speed**]. It is a rather theoretical calculation but it gives a simplified overview how the engine is connected to the wheels through the gearbox and the differential. The setup can be seen on Figure 1.2.

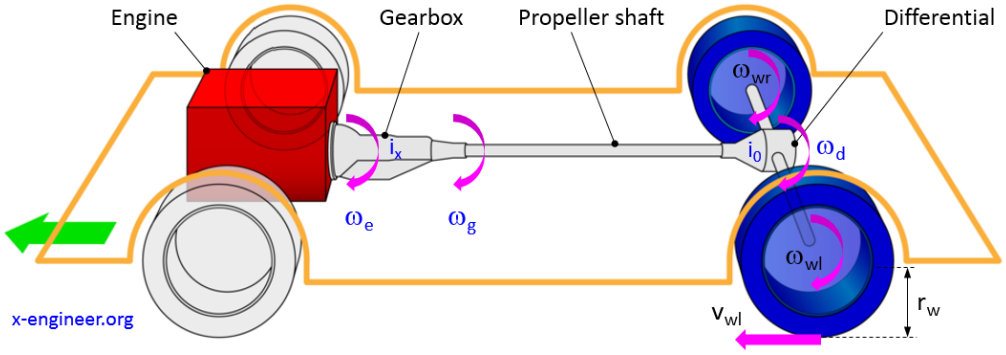


Figure 1.2: Calculating vehicle speed from engine speed [**engine_speed**]

The formula for determining the vehicle speed is the following:

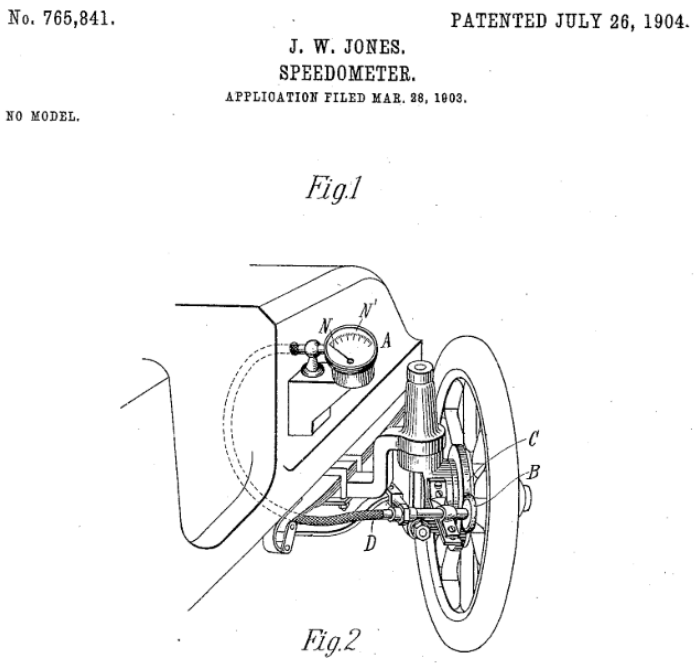
$$v_v = \frac{N_e \cdot \pi \cdot r_\omega}{30 \cdot i_x \cdot i_0} \left[\frac{m}{s}\right], \tag{1.3}$$

where N_e [rpm] is the engine speed, r_ω [m] is the wheel radius, i_x [-] is the gear ratio of the engaged gear and i_0 [-] is the gear ratio of the differential.

The main advantage of this method comes from its simplicity. Based on a one measured data, that is the engine shaft's rotational speed (N_e), the longitudinal vehicle speed can be determined. Contrary to this it can only give a estimate due to the simplifications it considers. It doesn't take in account the slip in the clutch between the gearbox and the engine shaft, the longitudinal and lateral slip of the tires, the difference in speed of the left and right wheels in case of traveling in a curve, the case when the gearbox and the engine are detached while the vehicle is being in neutral (gear 0) and when the wheels are locked but the vehicle is still motion. Considering the points mentioned above, this method could be used in highway driving where the vehicle travels with constant high speed on a straight path with little curvature.

1.2.2 Wheel speed

Following the previous method where the measured unit was the rotational speed of the engine's shaft, here the shift goes to measuring the rotational speed of the vehicle's wheel. This is done by a speedometer. The first speedometer is credited to Josip Belušić [**Josip_Belusic**]. His invention in 1888 was based on the electromagnetic induction which worked the following way. A flexible rotating cable is connected to the vehicle wheel's rotating shaft via a friction disk or a pair of gears that translates the rotational motion to a disk shaped magnet. This rotating magnet creates a fluctuating magnetic field which induces a rotating movement in the speed cup that surrounds the magnet. However the speed cup's movement is limited by a spring. This spring will allow greater rotation when greater torque applies to the speed cup that is in the case of faster rotation from the magnet. The needle that showed the speed of the vehicle is linked to the speed cup. This way the needle moved proportionally to the speed of the vehicle [**speedometer**]. Later this principle was patented by Otto Schulze in 1903 [**speedometer_patent_Otto**] and Joseph W Jones in 1904 [**speedometer_patent_Joseph**]. Both of the patents share the same principle.



No. 765,841. PATENTED JULY 26, 1904.

J. W. JONES.

SPEEDOMETER.

APPLICATION FILED MAR. 28, 1903.

NO MODEL.

Fig.1

Fig.2

Figure 1.3: Speedometer patented by Joseph W Jones in 1904 [**speedometer_patent_Joseph**]

The rather mechanical method described above was in use for several decades but it has multiple downsides: wear of the elements due to the mechanical connections, uncertainty coming from the mechanical measurement method and the fact of having the speed from only one wheel. To compensate the points mentioned before, electric wheel speed sensors were introduced to enhance the precision of the measurements. There are two types of

electronic wheel speed sensors: passive and active.

The principle of the passive wheel speed sensor is the following. A toothed reluctant ring is mounted on the rotating wheel. Above this ring a variable reluctance (VR) sensor is placed. When the wheel starts to rotate each tooth changes the magnetic flux through the coil of the sensor which induces an alternating voltage signal. The frequency of this analog sine wave is proportional to the rotational speed. However, the active wheel speed sensor uses a semiconductor magnetic sensor (Hall or magneto-resistive sensor) and a magnetic encoder ring attached to the rotating wheel. As the ring rotates, the magnetic field changes are detected by the magnetic sensor. These changes are then converted to square wave digital signals [**wheel_speed_sensors**]. The active sensors can overall provide a more precise measurement than the passive sensors.

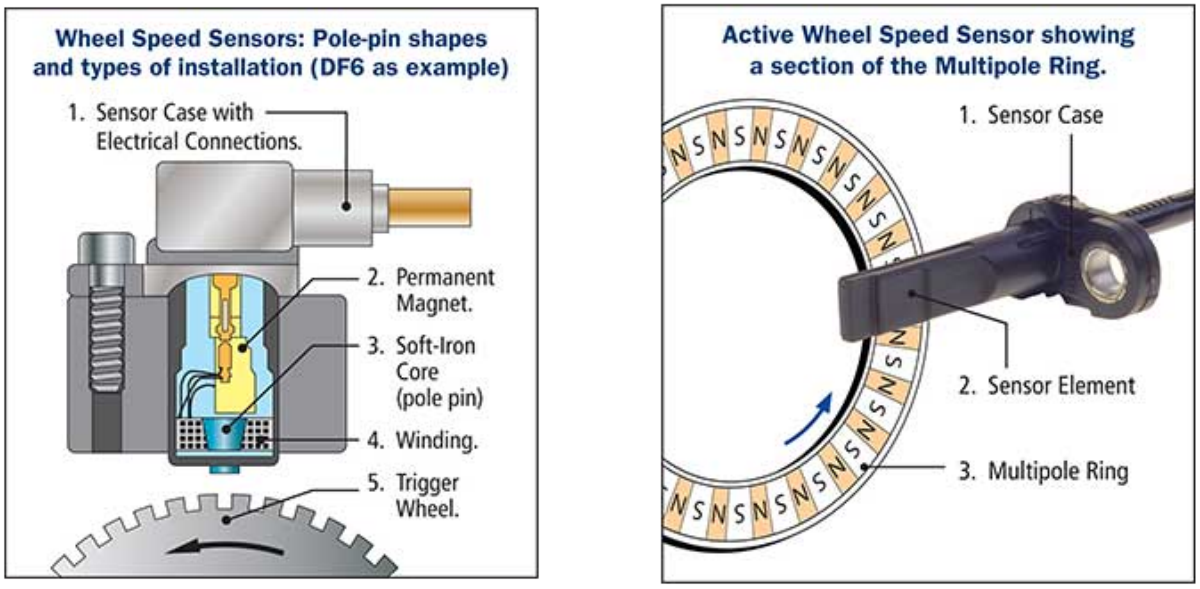


Figure 1.4: Passive (left) and active (right) wheel speed sensors from [**wheel_speed_sensors**]

In the case of these wheel speed sensors, the output is a rotational speed. From this rotational speed the ground speed of the specific wheel can be determined with the help of the diameter of the wheel. This measurement can be done for all the wheels, which will serve as an input for later discussed estimator algorithms.

1.2.3 Inertial Measurement Unit

The inertial measurement unit (IMU) can also provide relevant measurement data to determine the vehicle speed. An IMU consist of a set of linear accelerometers and gyroscopes to determine the acceleration along and the rotational rate around the longitudinal, lateral and vertical axes. This gives two vectors: acceleration **a** and angular velocity vector

$\boldsymbol{\omega}$ in the body fixes coordinate system:

$$\mathbf{a} = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} \text{ and } \boldsymbol{\omega} = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \tag{1.4}$$

Before calculating the velocity, the gravitational vector has to be deducted from the measured acceleration vector that can be done with the help of the angular velocity vector. From here, as the velocity vector's time derivative equals the acceleration vector, the velocity can be derived by an integral:

$$\mathbf{v}(t) = \mathbf{v}(t_0) + \int_{t_0}^{t} \mathbf{a}(\tau) d(\tau) \tag{1.5}$$

It is important to mention here that this measurement method is not robust by itself. Apart from setting the initial velocity of the vehicle, the biases in the accelerometers lead to integration drift that accumulates into large errors over time. To mitigate this issue this method has to be combined with other velocity measurements and estimators.

1.2.4 Global Navigation Satellite System

Global Navigation Satellite System (GNSS) refers to any constellation that provides global positioning, navigation and timing services. The currently available GNSS systems in Europe are GPS, Galileo, Glonass and BeiDou [**GNSS**].

The velocity of the vehicle can be obtained by exploiting the raw Doppler, the carrier phase or the pseudorange measurements with a GNSS receiver. These measurements are the following:

- Raw Doppler (RD) measurement:
  - The RD measurement captures how much does the frequency of the signal change between transmitting and receiving.

$$D = f_{transmitted} - f_{received} \tag{1.6}$$

  - The Doppler shift D is related to the change of the distance between the satellite and the receiver. From the Doppler shift the range rate $\dot{\rho}$ can be determined that is the radial velocity v_r along the range between the satellite and the receiver.

$$\dot{\rho} = v_r = \lambda \cdot D = \dot{\rho} + c \tag{1.7}$$

- Carrier phase measurement:

- Pseudorange differentiation:

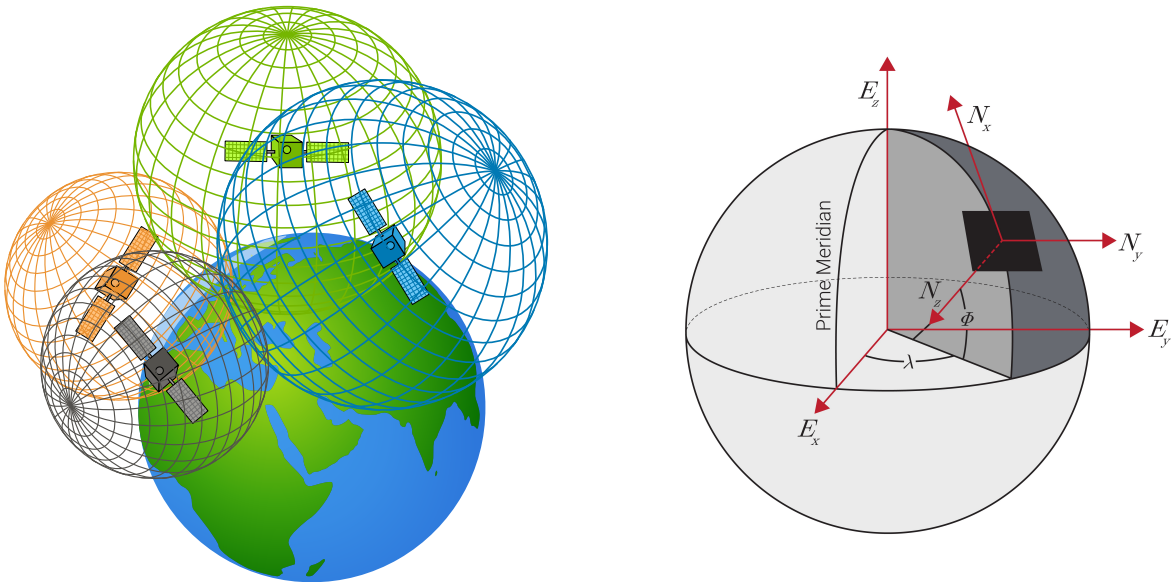


Figure 1.5: Trilateration (left) from [**gps-satellites-trilateration**] and ECEF coordinate system (right) from [**ECEF_coordinate_sys**]

1.3 Estimating vehicle speed

1.3.1 Accelerometer and speedometer

The simplest method to estimate the vehicle speed is to take the measurements from the speedometers at all the wheels. The longitudinal speed of each wheel can be determined from the rotational speed data from the sensors. These values then can be transformed into the CoG of the vehicle. There the transformed longitudinal velocities can be compared and the largest one can be chosen.

$$v_{CoG} = max[v_{FR}; v_{FL}; v_{RR}; v_{RL}], \tag{1.8}$$

where v_{FR}, v_{FL}, $v_{RR}$and v_{RL} are the transformed velocity values from each wheel.

Another simple algorithm for vehicle speed estimation is presented in [**lit_simple_veh_speed**]. Here the vehicle speed is determined by the measurements from the IMU's longitudinal accelerometer and the wheel speed sensors. The formula that estimates the speed is described as follows:

$$v_{CoG}(k) = v_{simple}(k_{nobrake}) + \sum_{i=k_{nobrake}}^{k} a_{meas}(i) \cdot dt, \tag{1.9}$$

where the k is the time step counter, $k_{nobrake}$ is the last time index where there was no

braking, a_{meas} is the acceleration value form the IMU's longitudinal accelerator and dt is the time sample. This method works in a way that estimation is taken apart into two categories based on whether the vehicle is barking or not. The simplicity comes at the price of inaccuracies. The estimator does not perform well when accelerometer offset is set and the dynamic change of the wheel radius is not implemented. An other difficulty comes from dealing with the noisy measurements from the wheel speed sensors and the IMU.

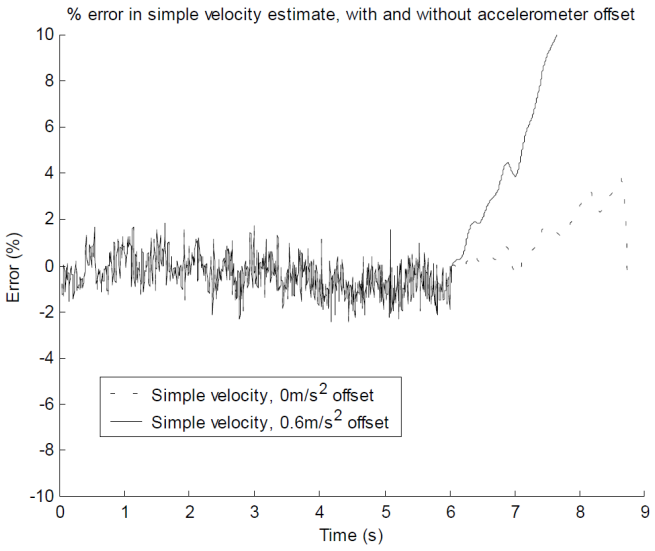


Figure 1.6: Simple algorithm for estimating vehicle speed form [**lit_simple_veh_speed**]

1.3.2 Kalman filter

To elevate the performance of the previously described methods, a Kalman filter can be added to the system. At a high level the Kalman filter estimates the vehicle speed from noisy sensor data by iterating between the following two steps:

1. Prediction
    - The filter takes the last estimate and predicts the next state's longitudinal or lateral velocity based on the simple dynamic motion model.
    - The filter provides an estimate and a uncertainty measure for the prediction.
2. Correction
    - Based on the predicted data and the new measurement data the filter calculates an optimal blend of the two values.
    - A posterior estimate is determined that is more accurate and less noisy than the pure measurement data.

Following this structure a Kalman filter is proposed in [**lit_for_Kalman_fuzzy**] to estimate the vehicle's speed. To enhance the accuracy of the estimator, 4 different driving situations are described with 4 different covariance matrices.

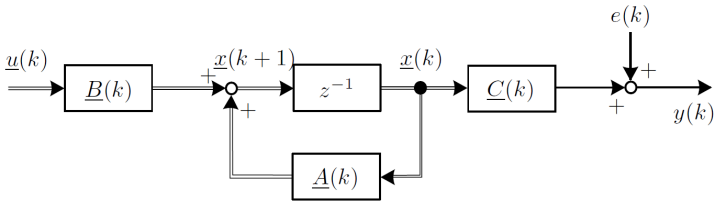


Figure 1.7: Vehicle speed estimation with Kalman filter from [**lit_for_Kalman_fuzzy**]

To elevate the usage of Kalman filter a more sophisticated and non-linear vehicle model can be used to predict the vehicle speed. In [**lit_model_based_est**] an Extended Kalman filter is introduced with vehicle model that runs parallel with the system itself. The overview of the filtering approach can be seen on figure 1.8

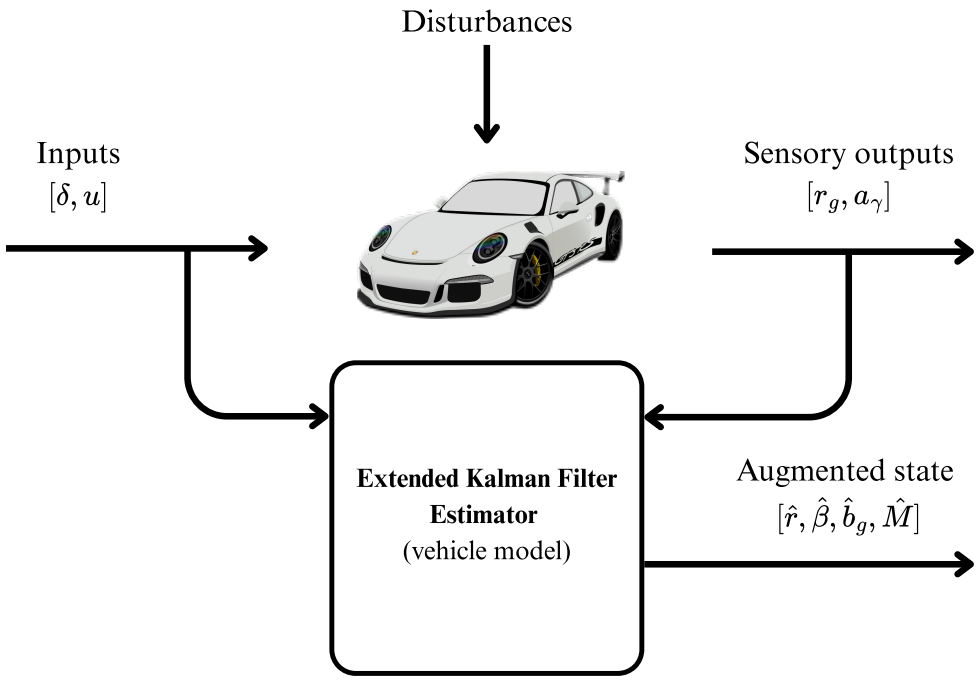


Figure 1.8: Extended Kalman filter for vehicle speed estimation from [**lit_model_based_est**]

1.3.3 Fuzzy logic

An other approach to estimate the vehicle speed is using fuzzy logic. Fuzzy logic offers heuristic approach to handle uncertainty and imprecision of the sensors. In [**lit_for_Kalman_fuzzy**] a comprehensive method is described using the Fuzzy logic to determine the vehicle speed using different driving scenarios.

The main motive behind the proposed estimator is determining the weights of the sensor signal values. All the speed values coming from the senors receive a weighting factor k_i. The weighted average is then formulated as follows:

$$\hat{v}_{CoG}(k) = \frac{\sum_{i=4}^{4} k_i \cdot v_{Ri,C}(k) + [\hat{v}_{CoG}(k-1) + T_S \cdot a_{X,C}(k)]}{\sum_{i=1}^{5} k_i}, \tag{1.10}$$

where the k_i weights are determined with the rules of the fuzzy logic. The overall scheme of the algorithm can be seen on figure 1.9.

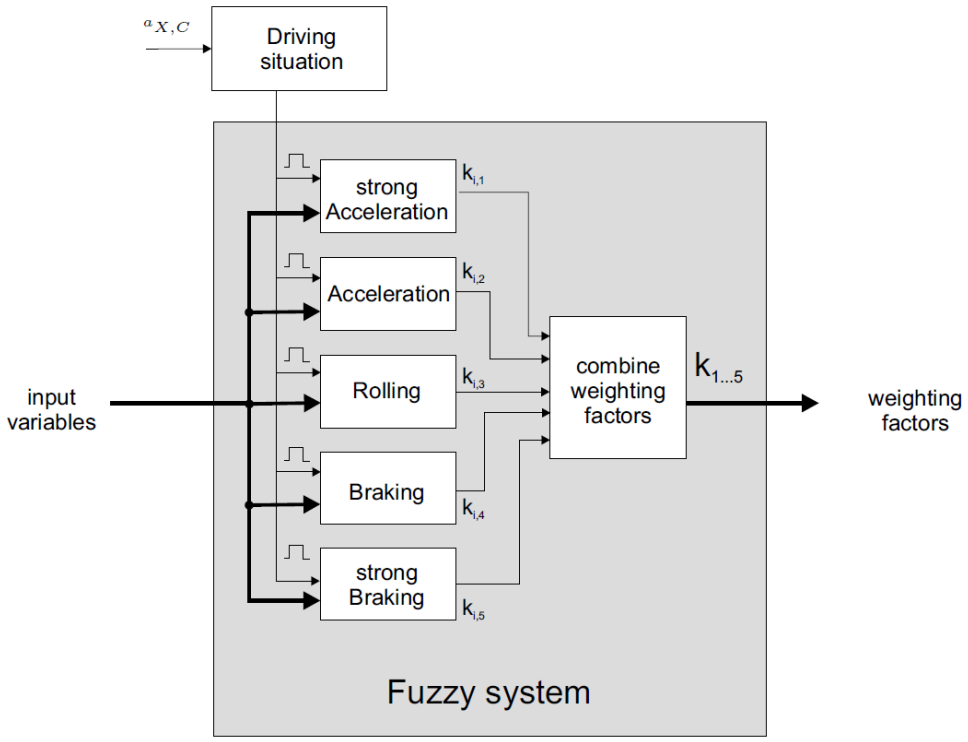


Figure 1.9: Vehicle speed estimation with Fuzzy logic from [**lit_for_Kalman_fuzzy**]

1.3.4 GNSS

There are multiple algorithms to determine the velocity of the vehicle using the GNSS system. In article [**GNSS_velocity_estimation**] 4 algorithms are described and compared. These methods are the following: Raw Doppler (RD), Time-Differenced Pseudorange (TDPR), Time-Differenced Carrier Phase (TDCP) and Double-Differenced Carrier Phase (DDCP). These algorithms were both tested in static and dynamic environments and postprocessed after the tests.

The most accurate algorithm in dynamic environments is the RD method that utilizes the Doppler shift from the carrier phase difference of the transmitted and received signal.

The measurement model can be written as:

$$\lambda D = \dot{\rho} + c(d\dot{t}_R - d\dot{t}_S) + \dot{I}_\rho + \dot{T}_\rho + \dot{\epsilon}_\rho, \tag{1.11}$$

where λ is the wavelength, D is the Doppler shift, $\dot{\rho}$ is the rate of the pseudorange, c is the speed of light in vacuum, $d\dot{t}_R$ and $d\dot{t}_S$ are the satellite and receiver clock drifts, $\dot{I}_\rho$ and $\dot{T}_\rho$ are the rates of change in the ionospheric and tropospheric delays and $\dot{\epsilon}_\rho$ combines the unmodeled error and observation noise. From this model the velocity of the vehicle can be determined in the global ECEF frame. The tests with this method resulted a [cm/s] level accuracy that is precise and accurate enough for driver assistance features.

The significant advantage of this algorithm is definitely the precision that it provides and can be used to enhance the previously described estimators. The disadvantages are that the GNSS receivers are not standard in passenger and commercial vehicles leaving little space for wide-spread usage of the advantages, the 10 [Hz] sampling time is relatively low that requires some type of interpolation between the samples and that the determined velocity is not in the vehicle's body fixed frame. For determining the longitudinal and lateral velocities the heading angle is required that can be calculated if two receivers are applied to the vehicle. The last disadvantage comes from the situations when the signals from the satellites are not visible in case of driving in a tunnel or urban areas with tall buildings.

1.3.5 Artificial Intelligence

After discussing the more conventional estimators that rely on deterministic formulas for calculating the vehicle speed, let the focus be shifted to the main topic of this thesis: Artificial Intelligence (AI) for vehicle speed estimation.

Artificial intelligence has been a greatly investigated field in the recent years for a wide range of applications. Despite the robustness and highly safety critical approach of the vehicle industry, the investigation of possible applications has been started. There are two main approaches:

1. Integrating AI algorithms to previously developed and used algorithms and models. This way an AI algorithm can be seen as a way to enhance the already developed and tested estimators.

2. Replace the previously used algorithms with a trained AI algorithm.

In article [**lit_deep_dynamics**] a physics-constrained neural network (PCNN) has been introduced. This approach merges the well established and developed physical models with the advantages of a neural network (NN). Here the created network is capable of learning the coefficients of a single track vehicle model that can predict the vehicle's

movement. The main advantage of this application is that by using the known dynamic and physical properties of a vehicle, the coefficients can be determined faster and more precisely than if there were no established constrains.

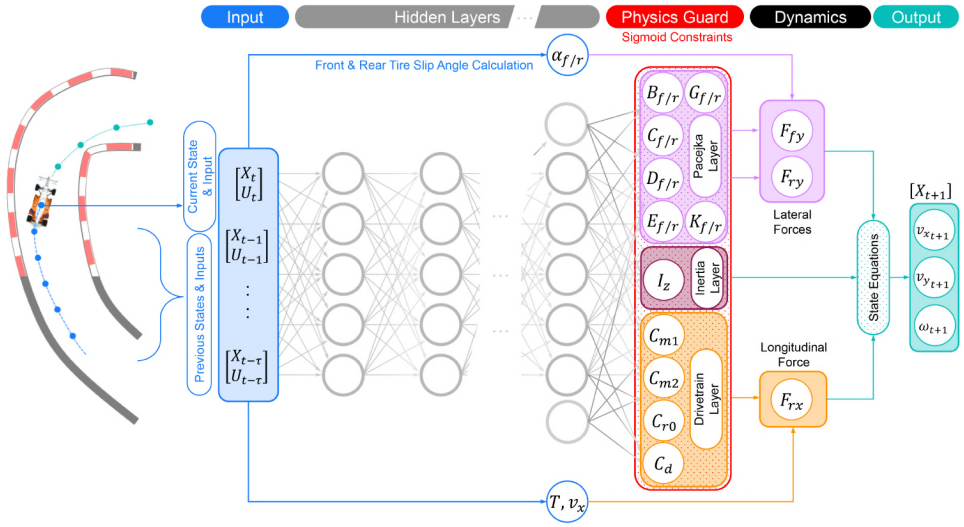


Figure 1.10: Physics constrained neural network from [**lit_deep_dynamics**]

A similar approach was investigated in [**lit_ai_physiscs_embedded_neural_network**] where a physics embedded neural network in combination of a vehicle model was developed where the previously discussed advantages were exploited. The outcome of this investigation was the performance shift from the baseline neural network model showing faster learning speed, higher accuracy and better generalization.

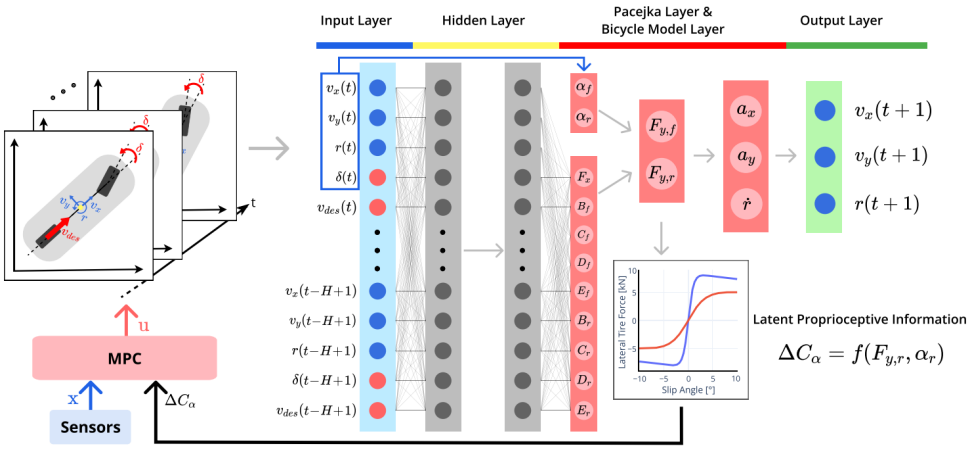


Figure 1.11: Physics embedded neural network for state estimation from [**lit_ai_physiscs_embedded_neural_network**]

An other approach is to combine AI algorithms with Kalman filtering to enhance state estimation [**lit_trans_lstm**]. Here a method for combining Long-Short Term Memory network (LSTM), Transformers and Expectation-Maximization - Kalman Filter (EM-KF). This combined approach of TL-KF (Transformers, LSTM - Kalman filter) is

capable of capturing long term dependencies and model time-series data. The structure of the developed model can be seen on figure 1.12

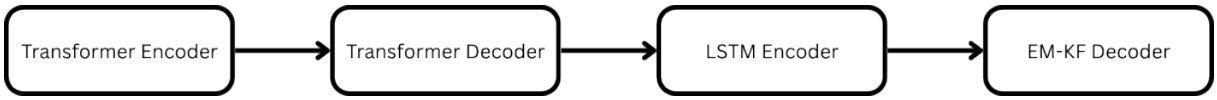


Figure 1.12: TS-KF model from [**lit_trans_lstm**]

1.4 Conclusions

During the literature review a comprehensive overview was presented about the used and developed vehicle speed estimators. It is visible that the well established estimators, like different model based estimators with Kalman filters have been there for longer time. In the meantime, the newly emerging field of AI has a promising future in state estimation aiding the already used estimator algortihms.

Chapter 2

Development environment

In this chapter the environment of the development will be described, where the structure of the vehicle and its sensors, and the requirements of the estimator will be discussed.

!! ezt pontosítani kell !!

2.1 Available sensor data

A modern vehicle is equipped with multiple ECU-s, numerous sensors and actuators that support and enhance driving quality. When considering vehicle motion control applications the available sensors are the following:

Sensor Type	Variables	Unit of Measurement
Wheel speed sensor	$\omega_{FR}, \omega_{FL}, \omega_{RR}, \omega_{RL}$	[°/s]
Drive torque sensor	$M_{FR}, M_{FL}, M_{RR}, M_{RL}$	[Nm]
Acceleration from IMU	a_x, a_y, a_z	[m/s^2]
Angular rates from IMU	$\dot{\alpha}$, $\dot{\beta}$, $\dot{\gamma}$	[°/s]
Brake pressure sensor	$P_{FR}, P_{FL}, P_{RR}, P_{RL}$	[bar]
Road wheel angles	$\delta_{FR}, \delta_{FL}, \delta_{RR}, \delta_{RL}$	[°]

Table 2.1: Overview of sensor types, variables, and units

It is important to note here that a the road wheels on one axle assign the same angle during normal driving conditions. Therefore the a road wheel angles ($\delta_{FR}, \delta_{FL}, \delta_{RR}, \delta_{RL}$) will be simplified to a two distinct values, where the front wheels will have the same angle ($\delta_{FR} = \delta_{FM}$ and $\delta_{FL} = \delta_{FM}$) and the rear wheels will have the same angle ($\delta_{RR} = \delta_{RM}$ and $\delta_{RL} = \delta_{RM}$). From this point the front and rear wheel angles will be represented as rwa$_{FM}$ and rwa$_{FM}$ respectively.

GNSS receivers can be also considered as precise sensors that are available during the development phase but not on the vehicles from the production line in order to keep the wider availability. Therefore the calculated longitudinal and lateral speeds from the GNSS

measurements will only serve for reference, validation, testing and training purposes. The mentioned sensor units can be seen on figure 2.1.

!! write about the normalization of the data !!

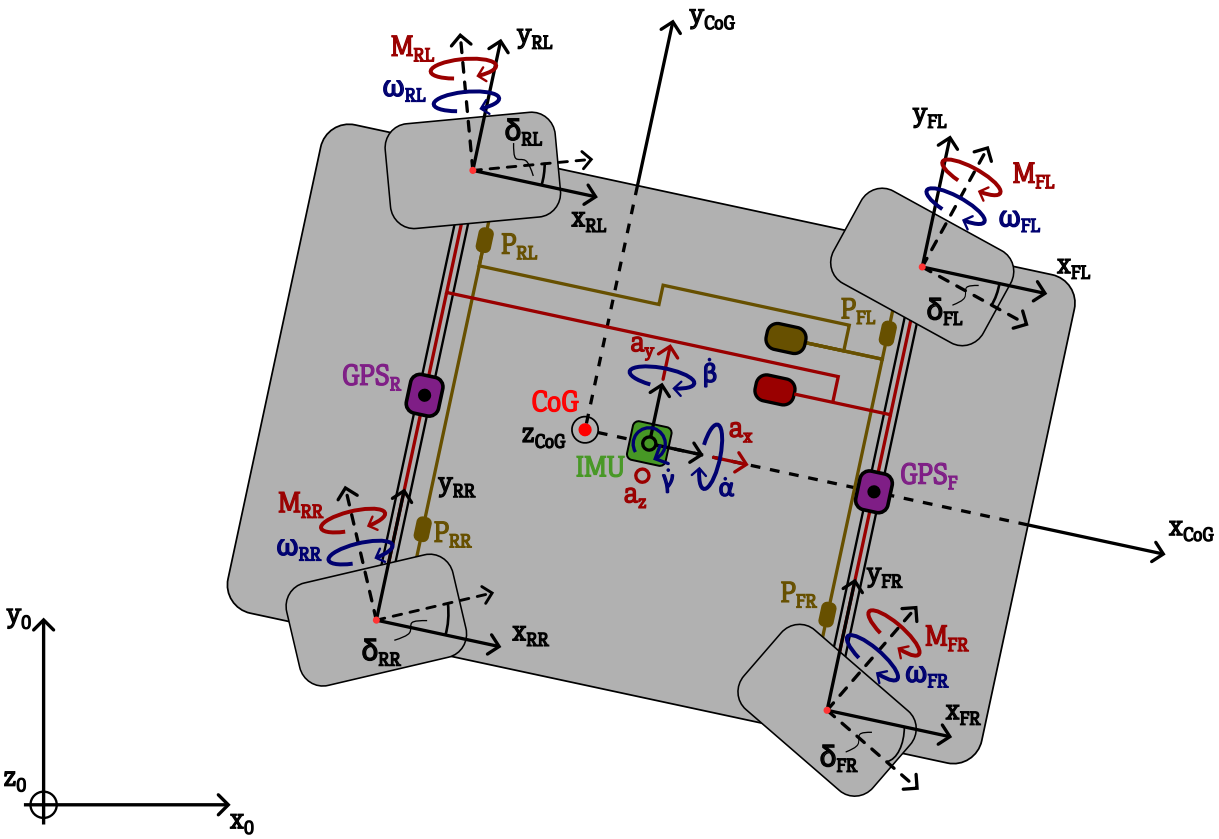


Figure 2.1: Available sensor data for speed estimation

The structure of the available sensor on the CAN bus data that was used in the further development is the following. Sampling time is 2 $[ms]$ for all the sensors except the speed values provided by the GNSS receivers, that is 100 $[ms]$. Above this, the sensor data is collected in a large enough buffer to provide all the values at the same time. This way at every 2 $[ms]$ all the inputs will arrive at the same time to the estimator. The speeds from the GNSS receivers will be handled separately from base structure. The structure can be seen in table 2.2:

time	a_x	ω_{FR}	ω_{FL}	ω_{RR}	ω_{RL}	M_{FR}	M_{FL}	M_{RR}	M_{RL}	a_y	a_z	$\dot{\alpha}$	$\dot{\beta}$	$\dot{\gamma}$	...
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	...
6.220	0.2819	31.3643	31.3791	31.5755	31.5894	0	0	19.087	19.087	0	0	0	0	0.2329	...
6.222	0.2819	31.3643	31.3791	31.5755	31.5894	0	0	19.087	19.087	0	0	0	0	0.2349	...
6.224	0.2819	31.3643	31.3791	31.5755	31.5894	0	0	19.087	19.087	0	0	0	0	0.2369	...
6.226	0.2819	31.3643	31.3791	31.5755	31.5894	0	0	19.087	19.087	0	0	0	0	0.2389	...
6.228	0.2819	31.3643	31.3791	31.5755	31.5894	0	0	19.087	19.087	0	0	0	0	0.2409	...
6.230	0.2893	31.511	31.5304	31.7222	31.7406	0	0	19.087	19.087	0	0	0	0	0.2429	...
6.232	0.2893	31.511	31.5304	31.7222	31.7406	0	0	19.087	19.087	0	0	0	0	0.2449	...
6.234	0.2893	31.511	31.5304	31.7222	31.7406	0	0	19.087	19.087	0	0	0	0	0.2469	...
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	...

Table 2.2: Example of available data structure with 2[ms] sampling time

2.2 Acquiring measurement data for development and testing

It is important to note that all the measurement data that was used during the development and testing phase of the estimators were recorded from a real vehicle. The vehicle used for the measurements was a luxury sedan. Apart from using the vehicle's built in sensors, two high precision GNNS receivers were installed on the roof of the vehicle to provide a reference speed, that can be used for validation and training purposes.

The measurement data was recorded in multiple test cases, where different driving scenarios were covered. The scenarios included basic maneuvers that were the following: accelerating until a target longitudinal acceleration is reached, decelerating until a target longitudinal acceleration is reached, steering until a target lateral acceleration is reached and a sine wave-like steering where the steering wheel is turned left and right continuously. These maneuvers were tested at different speeds to cover a wide range of operation. In addition the first two maneuvers can be split into two subgroups based on a change in a parameter that influenced the vehicle dynamics. With this in mind the test cases can be groupped into 6 different categories.

All together 43 measurements were recorded, where each measurement file contains a specified maneuver with a duration between 15 and 25 [s]. The sampling time of the recorded data is 2 [ms] but was downsampled to 10 [ms] during the development phase as the GNSS receiver only provided speed values with this sampling time.

In order to be able to train, validate and test the developed estimators, the recorded measurement data was split into three different sets. The training set contains 31 measeurement files, the validation set and the test set contain 6-6 measurement files. The splitting was done in a way that all the different maneuvers are represented in each set.

Besides the sensor data from the vehicle and the high precision GNSS receivers, the speed estimates from the vehicle's built in ECU were also recorded. These speed values will also serve as a reference for the performance of the developed estimators.

2.3 Ways to use AI to estimate vehicle speed

Before advancing to the possible algorithms for vehicle speed estimation using AI, some high level requirements shall be set to be able to validate the results. The requirements are the following:

1. The developed estimator shall provide more accurate vehicle speed values than a model based vehicle speed estimator.
2. The developed estimator shall be tested and validated with data that was not used for training purposes.
3. The developed estimator shall be trained with data that covers the range of use of a vehicle.
4. The developed estimator shall be applicable in real time usage in a test vehicle.

With these requirements in mind the possible solutions can be discussed. The algorithms can be categorized into two groups. Solutions where the vehicle speeds are directly estimated by the AI algorithm from the sensor input data (1) and where the AI algorithm is combined with an already existing estimator (2). Both of these solutions offer great potential in estimating vehicle speed.

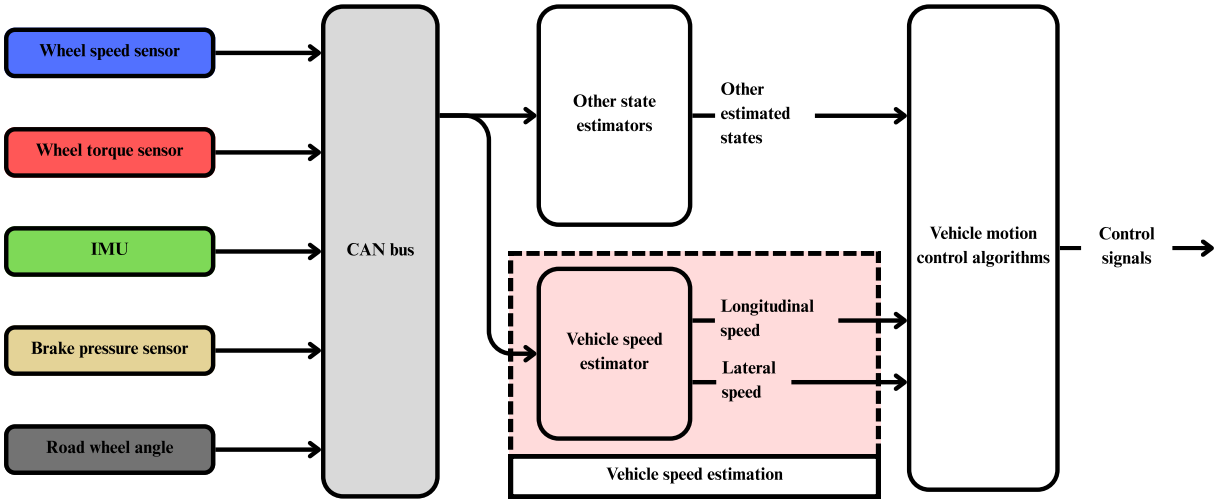


Figure 2.2: Vehicle speed estimation in the simplified control

2.3.1 Direct estimation

When directly estimating any state of the vehicle, the structure is the following. The trained AI model receives the sensor data as inputs and outputs the state for which it was trained. In this thesis the state to estimate is the vehicle's longitudinal and lateral speed.

The question now arises what models are suitable for this application? The possible models to use in this scenario are models that can handle time series data well and can estimate an output based on the sequence it was provided with. The possible models are the following:

- Basic/simple Recurrent Neural Network (RNN)
- Long-short Term Memory (LSTM)
- Gated Recurrent Unit (GRU)
- Transformer for time series
- Temporal Convolutinal Network (TCN)

The teaching phase of these models will be relatively similar. All models will be taught on previously recorded sensor data coming from vehicle simulations and real-life vehicle tests. These recorder datasets will be extended with a longitudinal and lateral speed values calculated from the GNSS measurements. This way every input will be labeled with the desired output.

2.3.2 Combination of existing estimators with AI model

As it can be seen in the literature review that there are already existing applications of combining an AI model with an already established state estimator. The key advantages of this approach could be improved robustness, adaptive parameter tuning of the applied filter with implementing AI and non-linear error compensation.

Some possible implementations can be the following:

- Residual learning:
  - A neural network predicts the residual error from a standard Extended Kalman Filter (EKF) applied for vehicle speed estimation. The final speed will result from the combination of the estimated and the neural network's value for the residual.
- Neural network aided Kalman filter:
  - The parts of the Kalman filter are replaced with trained modules
  - E.g when the predict module is replaced by a neural network that determines the next state.
- Adaptive gain scheduler:
  - The AI module dynamically tunes the noise covariance of the implemented Kalman filter based on the CAN signals.

2.3.3 Advantages and disadvantages of AI application

Prior to moving on to describing the developed models and estimators it is important to state the advantages and disadvantages when applying an AI model to a vehicle's control system. In case of a vehicle that is designed to be in contact (to transport or be driven by) with humans, the developed control system has to comply with specific international standards (e.g ISO 26262) that assures the safety of the system.

The main advantage when implementing AI to a vehicle control system is the possibility of capturing non-linear relationships between the CAN signals to estimate an arbitrary state. This property makes it possible to establish complex relationships between state variables without the knowledge of more complicated physical formulas.

On the other hand, there are some disadvantages that need to be considered. An implemented AI can be considered as a black box where the inner operation makes it difficult to validate from safety relevant perspective. This also creates a part of the system that lacks transparency and explainability. An other disadvantage may arise from missing out on data quality and availability. Resource constraints in embedded systems may also block usability if the created model uses more computational power or memory than the available amount.

Chapter 3

Direct estimation with AI algorithms

In this chapter the chosen AI models that are suitable for vehicle speed estimation will be discussed. First, three different versions of the recurrent neural network (RNN) will be presented including the simple recurrent neural network (sRNN), the long short-term memory network (LSTM) and the gated recurrent unit network (GRU). Then the temporal neural network (TCN) will be introduced as a convolutional alternative to the recurrent networks. Finally, transformer based models for time series will be discussed as another possible solution for vehicle speed estimation.

3.1 General development

Throughout this chapter different AI models will be presented for a sequence modeling task, that is estimating the vehicle speed from a time series of sensor measurements. The main goal is to find a a function that maps the input sensor data sequence to the desired vehicle speed value as accurately as possible.

$$\hat{y}_t = f(x_{t-n}, x_{t-n+1}, ..., x_t), \tag{3.1}$$

where $\hat{y}_t$ is the estimated vehicle speed at time step t, $x_{t-n}, x_{t-n+1}, ..., x_t$ are the input sensor measurements from time step $t-n$ to t, and f is the function approximated by the AI model.

The development of the different AI models for direct vehicle speed estimation will follow a similar structure in order to be able to compare the results and performance of the different architectures.

Initially, the train, validation and test datasets are the identical for all training sequences to ensure the comparibility of the architectures.

Next, the parameters of the architectural structure will be defined. The permutation stemming from all the possible parameter values cannot be tested due to the high computational and time cost. Therefore a grid like search will be performed where the most

influential parameters will be varied and the best performing combination will be chosen for further testing.

Finally a testing procedure will be carried on all the models using the previously unseen test dataset. The estimated vehicle speed values will be compared to the reference values where a set of metrics will be calculated to determine the performance of the different architectures.

The steps of the development procedure will be discussed in more detail in the section 3.2.1 containing the simple RNN model.

3.2 Recurrent Neural Network - RNN

3.2.1 Simple RNN

Theoretical background

Contrary with regular the feedforward neural networks where the output is determined only by the current values of the inputs, the Recurrent Neural Network (RNN) is designed in a way that after each step the information is fed back to the network. This way the output will be dependent on the previous inputs as well and the model will have a memory like property giving the possibility of handling temporal data.

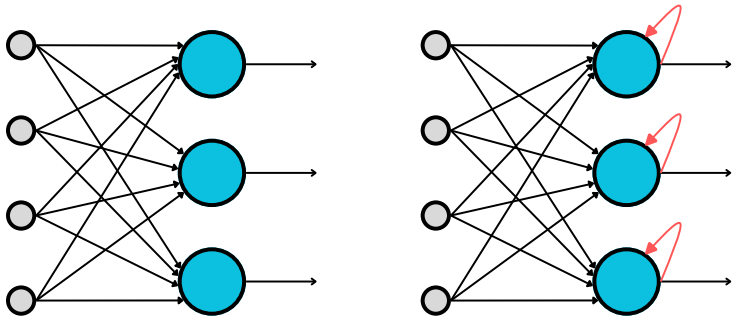


Figure 3.1: Feedforward nn (left) compared to recurrent nn (right)

The fundamental building blocks of a RNN are the recurrent units (RU) or RNN cells. These units hold an inner state that connects the in- and outputs. The inner states maintain information about previous inputs in a sequence. To understand how does the network's architecture function, let it be unfolded in time. This can be seen on figure 3.2.

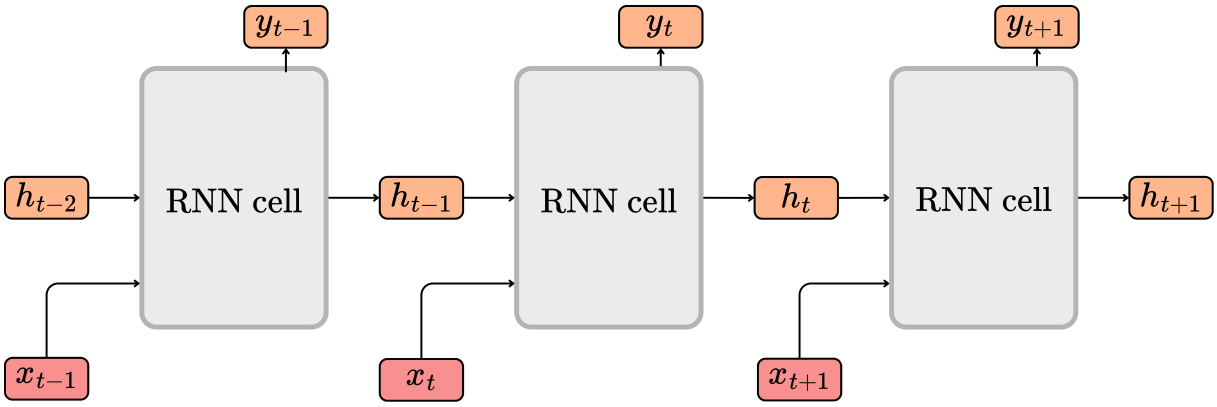


Figure 3.2: Unfolding the architecture of a RNN in time

The dimensions of the vectors in the RNN are the following:

- Input vector: $x_t \in \mathbb{R}^{d_x}$, where d_x is the size of the input vector.
- Hidden state: $h_t \in \mathbb{R}^{d_h}$, where d_h is the number of hidden units in the RNN layer.
- Output vector: $y_t \in \mathbb{R}^{d_y}$, where d_y is the size of the output vector.

It can be seen that at each time step the output vector (y_t) is determined by not only the input vector (x_t), but the state of the hidden layer in the previous time step (h_{t-1}) as well. The inner structure of the RNN cell can be seen on figure 3.3.

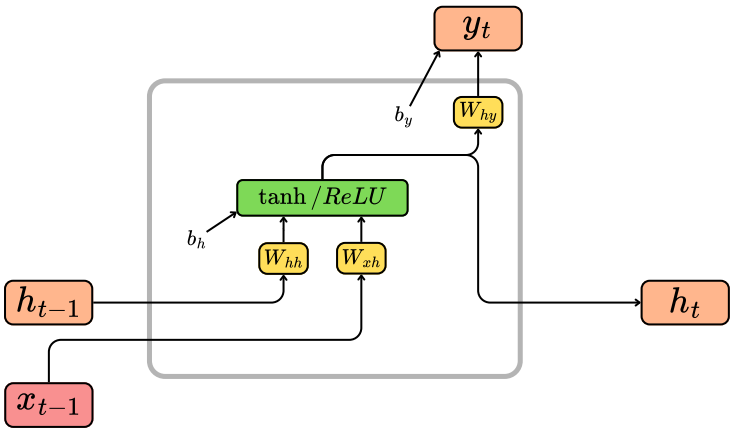


Figure 3.3: Recurrent Unit in the RNN

The dimensions of the weight matrices and bias vectors are the following:

- Weight matrix for the input vector: $W_{xh} \in \mathbb{R}^{d_x \times d_h}$
- Weight matrix for the state vector: $W_{hh} \in \mathbb{R}^{d_h \times d_h}$
- Weight matrix for the output vector: $W_{hy} \in \mathbb{R}^{d_h \times d_y}$
- Bias vector for the hidden layer: $b_h \in \mathbb{R}^{d_h}$

- Bias vector for the output layer: $b_y \in \mathbb{R}^{d_y}$

The governing equations for the RNN cell are the following:

1. Hidden state update 3.2: The hidden state at time step t is computed using the current input vector (x_t) and the previous hidden state (h_{t-1}) through a weighted sum followed by an activation function (e.g., tanh or ReLU).

$$h_t = f(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h) \tag{3.2}$$

2. Output computation 3.3: The output vector at time step t is determined from the current hidden state (h_t) using a linear transformation followed by an activation function (depending on the task, e.g., softmax for classification or identity for regression).

$$y_t = g(W_{hy} \cdot h_t + b_y) \tag{3.3}$$

To determine the weights and biases of the network, backpropagation through time (BPTT) is used, that is an extended version of the backpropagation used in feedforward neural networks (FNNs). This method is based on the error between the determined output of the network and the desired/labeled output at time step t. BPTT involves two major steps: forward pass and backward pass. During the forward pass the inputs are passed through the network to compute the outputs and the loss, while during the backward pass the gradients of the loss with respect to the weights are calculated to update them accordingly. The steps of BPTT are the following:

1. Forward pass

   - During the forward pass the sequence of inputs form $t = 1$ to $t = n$ passes through the network, where n is the length of the sequence.
   - The output vector y_t is determined at every timestep with the use of the previously described formulas.
   - The loss is determined at every timestep between the desired (y_t^d) and the estimated (y_t) output.
   - The loss function is given by the mean squared error (MSELoss):

$$L(y, y^d) = \frac{1}{t} \sum_{t=1}^{n} (y_t - y_t^d)^2 \tag{3.4}$$

2. Backward pass

   - During the backward pass the gradients of the loss function $L(y, y^d)$ are calculated with respect of the weights of the network (w_{xh}, W_{hh}, W_{hy}).

- W.r.t the weight matrices the derivatives of the loss functions can be determined as:

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^{n} \frac{\partial L}{\partial h_t} \cdot \frac{\partial h_t}{\partial W_{xh}} \tag{3.5}$$

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{n} \frac{\partial L}{\partial h_t} \cdot \frac{\partial h_t}{\partial W_{hh}} \tag{3.6}$$

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^{n} \frac{\partial L}{\partial y_t^d} \cdot \frac{\partial y_t^d}{\partial W_{hy}} \tag{3.7}$$

- The gradient values inform about the direction (increase or decrease) and the magnitude of the change in the weight matrices.
- These gradients can then be used for updating the values of each weight matrix with the help of the chosen optimizer (e.g. SGD, Adam).

Common issues when applying RNNs are exploding and vanishing gradients during the backward pass. Both of these refer to the magnitude of the calculated gradients: $\left\| \frac{\partial L}{\partial W} \right\|$. Exploding gradient problem stems from the gradient being too high which is a consequence of the learning rate being to big. On the other hand vanishing gradient refers to a gradient being near zero, which leads to slow learning.

Implementation and results

The objective is to find the best performing RNN architecture for estimating the vehicle speed from the given input data. For this the following paramaters were taken into account: number of inputs, hidden size and number of layers. The number of inputs refers to the dimension fo the input vector (d_x) where every element of the vector corresponds to a different sensor measurement. The hidden size refers to the dimension o the hidden state vector (d_h) and it determines how much information the hidden state can store. Finally the number of layers refers to how many RNN layers are stacked on top of each other. An example for a 2-layer RNN can be seen on figure 3.4. Another parameter that was considered, but is not part of the architecture was the sequence length. This parameter defines how many time steps the network will look back when estimating the current vehicle speed.

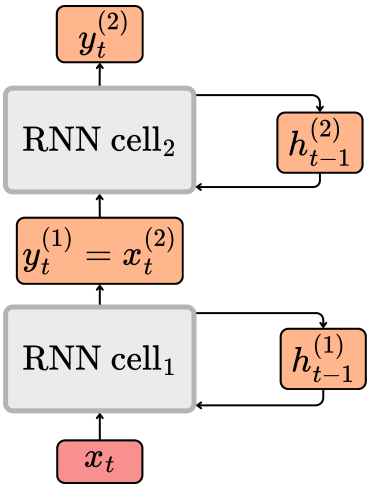


Figure 3.4: 2-layer RNN architecture

The initial values for the mentioned hyperparameters can be seen in table in 3.1. The values were chosen based on common practices in literature and previous experience with RNN-s. A grid search was performed over the hyperparameter space to find the best combination of values that yield the lowest validation loss. The training was done using the Adam optimizer with a learning rate of 0.001 and a batch size of 128. The models were trained for 100 epochs, and early stopping was implemented to prevent overfitting.

Hyperparameters	**Possible values**
Number of inputs (d_x)	23; 12
Hidden size (d_h)	64; 96; 128; 192
Number of layers	1; 2
Sequence length	50; 100; 150; 200

Table 3.1: Hyperparameter ranges for RNN architecture search

First, the effect of the number of inputs was examined with the intent of reducing the complexity of the model. The available sensor measurements at each time step can be found in table 2.1. When considering all the available sensor measurements the input vector has a dimension of 23. However, the number of inputs can be reduced to a subset of all the available values when considering only the most relevant ones. Table 3.2 contains the subset of the inputs that was used. Within the table x marks the inputs that were used and - marks the ones that were omitted in the subset.

Sensor measurement	**ID 1**	**ID 2**	**Reason of omiting**
Wheel speed FL - ω_{FL}	x	x	-
Wheel speed FR - ω_{FR}	x	x	-
Wheel speed RL - ω_{RL}	x	x	-
Wheel speed RR - ω_{RR}	x	x	-
Drive torque FL - M_{FL}	x	x	-
Drive torque FR - M_{FR}	x	-	The drive torques for the wheels on one axle is equivalent $\rightarrow$ no additional info compared to M_{FL}
Drive torque RL - M_{RL}	x	x	-
Drive torque FL - M_{RR}	x	x	The drive torques for the wheels on one axle is equivalent $\rightarrow$ no additional info compared to M_{RL}
Long. acceleration - a_x	x	x	-
Lat. acceleration - a_y	x	x	-
Vert. acceleration - a_z	x	-	Vertical acceleration is not significant as the vehicle measurements were made on a plane
Roll rate - $\dot{\alpha}$	x	x	Roll rate is significant in a planar measurement but was omitted to imitate a 2D vehicle
Pitch rate - $\dot{\beta}$	x	-	Pitch rate is not significant as the vehicle measurements were made on a plane surface
Yaw rate - $\dot{\gamma}$	x	x	-
Brake pressure FR - P_{FR}	x	x	-
Brake pressure FL - P_{FL}	x	-	Brake pressures for the wheels on one axle is equivalent $\rightarrow$ no additional info compared to P_{FR}
Brake pressure RR - P_{RR}	x	x	-
Brake pressure RL - P_{RL}	x	-	Brake pressures for the wheels on one axle is equivalent $\rightarrow$ no additional info compared to P_{RR}
Road wheel angle - rwa_{FM}	x	x	-
Road wheel angle - rwa_{FL}	x	-	Rear wheel steering was switched off during the measurements

Table 3.2: The set of sensor measurements used as inputs

The hidden sizes, the number of layers and the sequence lengths were varied and a permutation of the possible values was tested. Two different input sets were used: one with all the available sensor measurements (23 inputs) and one with the reduced set of sensor measurements (12 inputs). This way the effect of the number of inputs on the performance of the model could be examined.

The chosen performance metrics for evaluating the models and the estimators were the root mean squared error (RMSE) and the mean absolute error (MAE). Apart from that, and absolute maximum error (AMaxE) metric was also calculated to see the worst case performance of the models. Furthermore a percentage within threshold (PWT) metric was also defined to see how many of the estimations were within a certain error threshold. The PWT metric is defined as follows:

$$TBM = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(|y_i^d - y_i| < \epsilon) \times 100\%, \tag{3.8}$$

where N is the total number of estimations, y_i is the estimated value, y_i^d is the desired value, ϵ is the error threshold (e.g., 1 km/h), and $\mathbb{I}$ is the indicator function that returns 1 if the condition is true and 0 otherwise. Throughout the development two different thresholds were considered: 0.3 km/h and 0.4 km/h. These will be denoted as PWT 0.3 and PWT 0.4 in the following tables.

After the training and validation procedure all the models were tested on the unseen test dataset. The code using for training and validating can be found at `AI_training/3_code/training_RNN.py`. The 2-2 best performing architectures were chosen based on the root mean squared error (RMSE) metric. The 2 best performing architectures with the two input sets can be seen in table 3.3. The results that can be found in table 3.4 show that the model with 12 inputs performs slightly better than the one with 23 inputs, indicating that the reduced input set was sufficient for accurate vehicle speed estimation.

	Input ID	**Sequence Length**	**Hidden size**	**Number of layers**
1	1	TBD	TBD	TBD
2	1	TBD	TBD	TBD
1	2	TBD	TBD	TBD
2	2	TBD	TBD	TBD

Table 3.3: Results of the best performing RNN architecture on the test dataset

	RMSE [$\frac{km}{h}$]	MAE [$\frac{km}{h}$]	AMaxE [$\frac{km}{h}$]	PWT 0.3 [%]	PWT 0.4 [%]
1	TBD	TBD	TBD	TBD	TBD
2	TBD	TBD	TBD	TBD	TBD
1	TBD	TBD	TBD	TBD	TBD
2	TBD	TBD	TBD	TBD	TBD

Table 3.4: Performance metrics of the best performing RNN architectures on the test dataset

A design decision from this point on was to continue the development with the reduced input set (12 inputs) as it yielded similar results while having a lower complexity.

After obtaining these results another grid search was performed around the 2 best performing architectures to see if further improvements could be achieved by finetuning the hyperparameters. The hyperparameter ranges for the finetuning can be seen in table 3.5.

Around 1st best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	
Number of layers	
Sequence length	
Around 2nd best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	
Number of layers	
Sequence length	

Table 3.5: Batch 2: Hyperparameter ranges for RNN architecture search

Following the new training and validation procedure the best performing architectures were again tested on the unseen test dataset. The results of the best performing architecture can be seen in table 3.6 and 3.7. It can be seen that the finetuning of the hyperparameters led to a slight improvement in the performance of the models.

	Input ID	Sequence Length	Hidden size	Number of layers
1	2	TBD	TBD	TBD

Table 3.6: Results of the best performing RNN architecture on the test dataset after finetuning

	RMSE [$\frac{km}{h}$]	MAE [$\frac{km}{h}$]	AMaxE [$\frac{km}{h}$]	PWT 0.3 [%]	PWT 0.4 [%]
1	TBD	TBD	TBD	TBD	TBD

Table 3.7: Performance metrics of the best performing RNN architecture< on the test dataset after finetuning

3.2.2 Long Short-Term Memory - LSTM

Theoretical background

The Long Short-Term Memory (LSTM) networks build on the structure of the RNN-s but introduce additional modifications to improve the performance in long term dependencies when handling sequential data. The main difference between the two structures are the introduction of memory cells and a series of so called gating mehanisms. The output of the LSTM unit is determined not only by the current input and the previous hidden state, as it was with the conventional RNN-s, but also by the newly introduced cell state that carries information across the time steps. The structure of an LSTM network unfolded in time can be seen on figure 3.5, where x_t is the input vector, h_t is the hidden state vector, C_t is the cell state vector and y_t is the output vector at time step t.

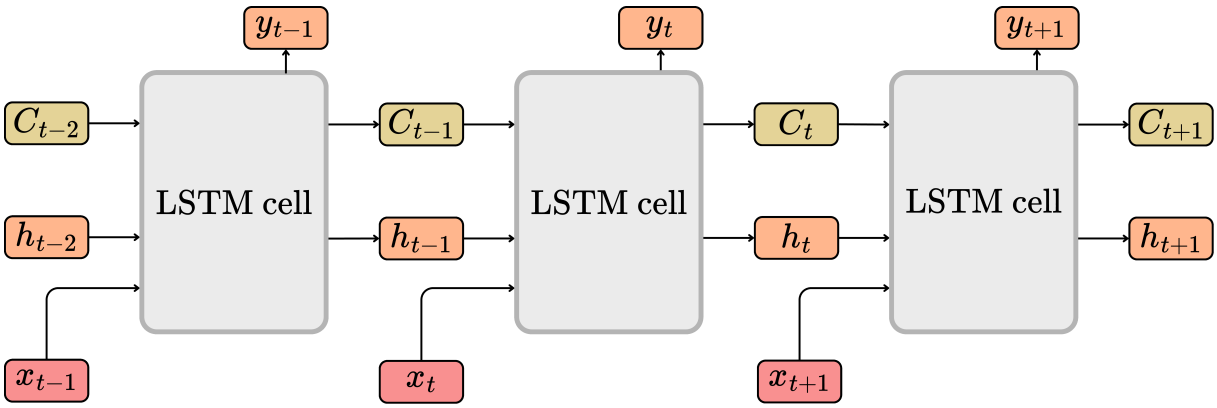


Figure 3.5: LSTM sequence unfolded in time

The dimensions of the previously mentioned vectors are the following:

- Input vector: $x_t \in \mathbb{R}^{d_x}$, where d_x is the size of the input vector.
- Hidden state: $h_t \in \mathbb{R}^{d_h}$, where d_h is the number of hidden units in the LSTM layer.
- Cell state: $C_t \in \mathbb{R}^{d_h}$, that has the same dimension as the hidden state vector.
- Output vector: $y_t \in \mathbb{R}^{d_y}$, where d_y is the size of the output vector.

I want to note here that I have not included the dimension of the batch size B for simplicity. Here I only consider the inputs to be a single sequence, as it will be when using the netowork for estimating the vehicle speed. When considering trainging with

batches of sequences, the dimensions would be: $x_t \in \mathbb{R}^{B \times d_x}$, $h_t \in \mathbb{R}^{B \times d_h}$, $C_t \in \mathbb{R}^{B \times d_h}$, $y_t \in \mathbb{R}^{B \times d_y}$.

The white-box represetation of the LSTM cell can be seen on figure 3.6, where the weight matrices and the bias vectors are also presented. The dimensions of these vectors are the following:

- Weight matrices for the input vector: $W_{xf}, W_{xi}, W_{xc}, W_{xo} \in \mathbb{R}^{d_x \times d_h}$
- Weight matrices for the state vectors: $W_{hf}, W_{hi}, W_{hc}, W_{ho} \in \mathbb{R}^{d_h \times d_h}$
- Weight matrix for the output vector: $W_{hy} \in \mathbb{R}^{d_h \times d_y}$
- Bias vectors for the operation: $b_f, b_i, b_c, b_o \in \mathbb{R}^{d_h}, b_y \in \mathbb{R}^{d_y}$

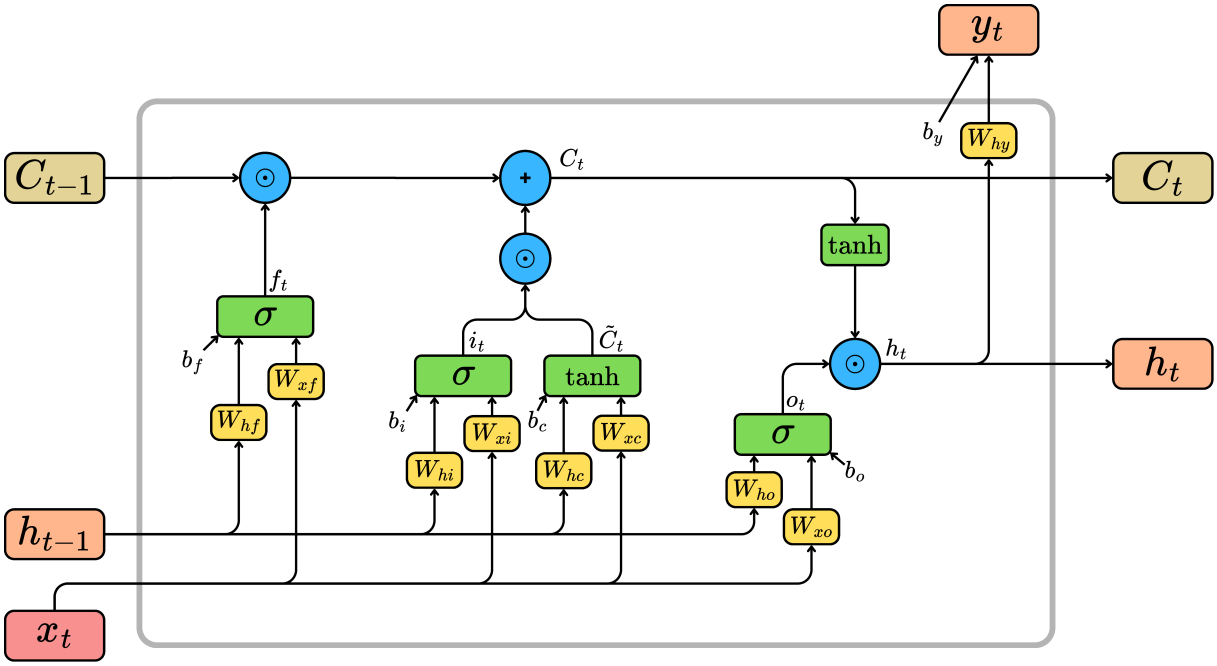


Figure 3.6: LSTM unit with the gating mechanisms

Throughout the LSTM cell the so called gates control the outputs, that are namely the forget gate (f_t), input gate (i_t) and the output gate (o_t). These are mainly functions that use either sigmoid or tanh activation functions. Furthermore the cell contains operations to determine the candidate cell state ($\tilde{C}_t$), the new cell state (C_t), the new hidden state (h_t) and the output (y_t). All the gates and operations can be described with the following equations:

1. Forget gate 3.9: The goal of the forget gate is to determine what information from the previous cell state (C_{t-1}) shall be kept and what shall be discarded. This is done by using a sigmoid activation function that outputs values between 0 and 1, where 0 means "completely forget" and 1 means "completely keep".

$$f_t = \sigma(W_{xf} \cdot x_t + W_{hf} \cdot h_{t-1} + b_f) \tag{3.9}$$

2. Input gate 3.10: The input gate determines what new information shall be added to the cell state. Similar to the forget gate, it uses a sigmoid activation function to decide which values to update.

$$i_t = \sigma(W_{xi} \cdot x_t + W_{hi} \cdot h_{t-1} + b_i) \tag{3.10}$$

3. Candidate cell state 3.11: This operation creates a vector of new candidate values ($\tilde{C}_t$) that could be added to the cell state. It uses a tanh activation function to ensure the values are between -1 and 1.

$$\tilde{C}_t = \tanh(W_{xc} \cdot x_t + W_{hc} \cdot h_{t-1} + b_c) \tag{3.11}$$

4. New cell state 3.12: The new cell state (C_t) is computed by combining the previous cell state (C_{t-1}) modified by the forget gate and the candidate cell state ($\tilde{C}_t$) scaled by the input gate. ($\odot$ denotes element-wise multiplication).

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{3.12}$$

5. Output gate 3.13: The output gate determines what part of the cell state will be output as the new hidden state. It uses a sigmoid activation function to decide which parts of the cell state to output.

$$o_t = \sigma(W_{xo} \cdot x_t + W_{ho} \cdot h_{t-1} + b_o) \tag{3.13}$$

6. New hidden state 3.14: The new hidden state (h_t) is computed by applying the output gate to the tanh of the new cell state.

$$h_t = o_t \odot \tanh(C_t) \tag{3.14}$$

7. Output 3.15: The output vector (y_t) is determined from the new hidden state using a linear transformation followed by an activation function (depending on the task).

$$y_t = W_{hy} \cdot h_t + b_y \tag{3.15}$$

After defining the computational steps withing an LSTM cell, that are also the steps of the forward pass, the training process can be discussed. The objective of the training sequence is to optimize the weight matrices and the bias vectors to minimize the defined loss ($\mathcal{L}$) over a dedicated training dataset, just as at the RNN-s. The steps of the backpropagation through time (BPTT) for LSTM networks are the following:

The question still arises why is this LSTM structure better at handling long term dependencies than the conventional RNN-s? The answer lies in the constant error carousel (CEC) property of the LSTM cell.

When multiple layers are stacked on top of each other to create a deep LSTM network...TO BE CONTINUED

Implementation and results

The implementation and training procedure of the LSTM network is similar to the one used for the RNN-s. The same reduced input set (12 inputs) was used here as well. A grid search was performed over the hyperparameter space to find the best combination of values that yield the lowest validation loss. The training was done in a similar manner as with the RNN-s. For the first batch of grid search the hyperparameter ranges are the as in table 3.1.

The results obtained after testing the created models on the same unseen test dataset . The code using for training and validating can be found at `AI_training/3_code/training_LSTM.py`. The 2 best performing architectures can be seen in table 3.8. The results that can be found in table 3.9 show that the LSTM models outperformed the RNN models, indicating that the LSTM architecture is better suited for capturing the temporal dependencies in the vehicle speed estimation task.

	Input ID	**Sequence Length**	**Hidden size**	**Number of layers**
1	2	TBD	TBD	TBD
2	2	TBD	TBD	TBD

Table 3.8: Results of the best performing LSTM architecture on the test dataset

	RMSE [$\frac{km}{h}$]	**MAE [$\frac{km}{h}$]**	**AMaxE [$\frac{km}{h}$]**	**PWT 0.3 [%]**	**PWT 0.4 [%]**
1	TBD	TBD	TBD	TBD	TBD
2	TBD	TBD	TBD	TBD	TBD

Table 3.9: Performance metrics of the best performing LSTM architectures on the test dataset

After evaluating these results another grid search was performed around the 2 best performing architectures to see if further improvements could be achieved by finetuning the hyperparameters. The hyperparameter ranges for the finetuning can be seen in table 3.10.

Around 1st best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	
Number of layers	
Sequence length	

Around 2nd best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	
Number of layers	
Sequence length	

Table 3.10: Batch 2: Hyperparameter ranges for LSTM architecture search

Following the training and evaluation of the models from the second batch of grid search, the best performing architecture was selected based on the RMSE metric on the test dataset. The results of this best model can be seen in table 3.11.

RMSE [$\frac{km}{h}$]	**MAE [$\frac{km}{h}$]**	**AMaxE [$\frac{km}{h}$]**	**PWT 0.3 [%]**	**PWT 0.4 [%]**
TBD	TBD	TBD	TBD	TBD

Table 3.11: Performance metrics of the best performing LSTM architecture from the second batch on the test dataset

3.2.3 Gated Recurrent Unit - GRU

Theoretical background

The Gated Recurrent Unit (GRU) network also builds on the structure of the original RNN-s but introduces a simplified architecture compared to the LSTM network. The GRU uses similar gating mechanisms to control the flow of the input and the state information, but omits the separate cell state, therefore making it a lighter alternative. The structure of a GRU network unfolded in time can be seen on figure 3.7, where x_t is the input vector, h_t is the hidden state vector and y_t is the output vector at time step t.

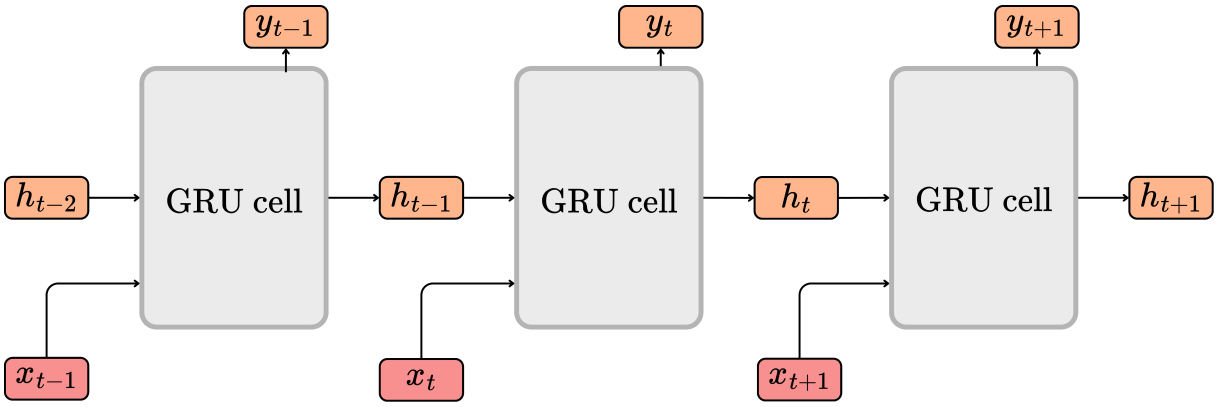


Figure 3.7: GRU sequence unfolded in time

The dimensions of the previously mentioned vectors are the following:

- Input vector: $x_t \in \mathbb{R}^{d_x}$, where d_x is the size of the input vector.
- Hidden state: $h_t \in \mathbb{R}^{d_h}$, where d_h is the number of hidden units in the GRU layer.
- Output vector: $y_t \in \mathbb{R}^{d_y}$, where d_y is the size of the output vector.

Batch size B is not included here for simplicity the same way as before, but when considering training with batches of sequences, the dimensions would be: $x_t \in \mathbb{R}^{B \times d_x}$, $h_t \in \mathbb{R}^{B \times d_h}$, $y_t \in \mathbb{R}^{B \times d_y}$.

The same way as with the LSTM cell, the white-box representation of the GRU cell can be seen on figure 3.8, where the weight matrices and the bias vectors are also presented. The dimensions of these vectors are the following:

- Weight matrices for the input vector: $W_{xr}, W_{xz}, W_{xh} \in \mathbb{R}^{d_x \times d_h}$
- Weight matrices for the state vectors: $W_{hr}, W_{hz}, W_{hh} \in \mathbb{R}^{d_h \times d_h}$
- Weight matrix for the output vector: $W_{hy} \in \mathbb{R}^{d_h \times d_y}$
- Bias vectors for the operation: $b_r, b_z, b_h, b_y \in \mathbb{R}^{d_h}, b_y \in \mathbb{R}^{d_y}$

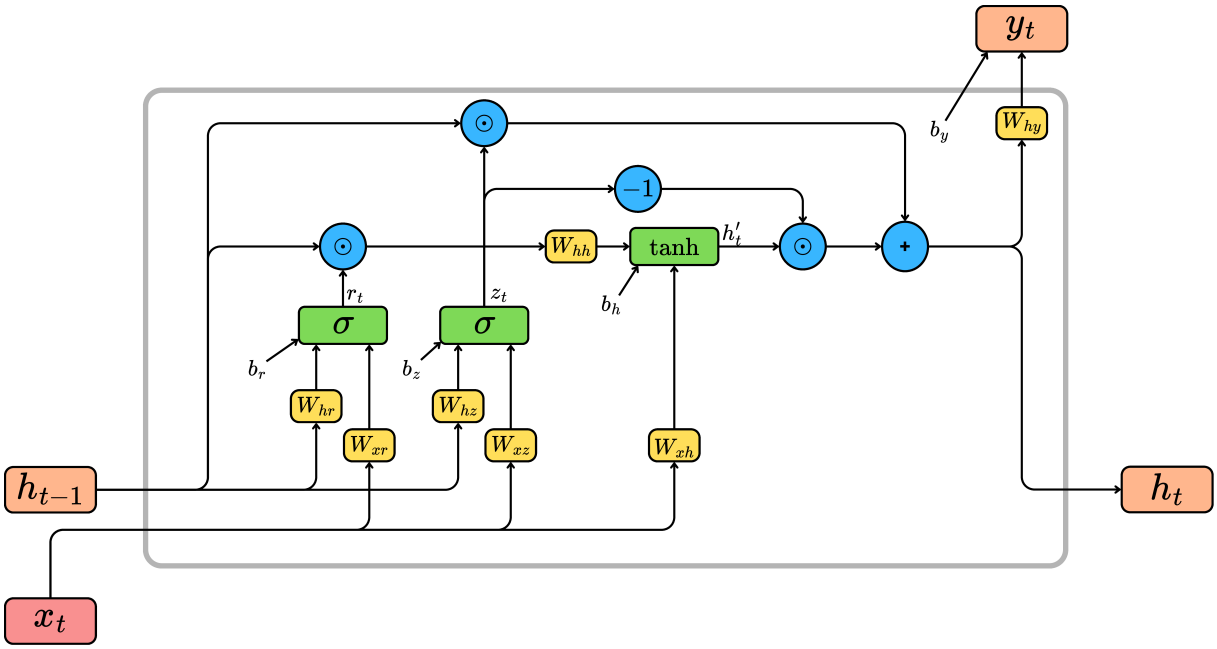


Figure 3.8: GRU unit with the gating mechanisms

The gates that are within the GRU cell are the reset gate (r_t) and the update gate (z_t). These are mainly functions that use either sigmoid or tanh activation functions. Furthermore the cell contains operations to determine the candidate hidden state ($\tilde{h}_t$), the new hidden state (h_t) and the output (y_t). All the gates and operations can be described with the following equations:

1. Reset gate 3.16: The reset gate determines how much of the previous hidden state (h_{t-1}) shall be forgotten when calculating the candidate hidden state.

$$r_t = \sigma(W_{xr} \cdot x_t + W_{hr} \cdot h_{t-1} + b_r) \tag{3.16}$$

2. Update gate 3.17: The update gate decides how much of the previous hidden state shall be carried forward to the new hidden state.

$$z_t = \sigma(W_{xz} \cdot x_t + W_{hz} \cdot h_{t-1} + b_z) \tag{3.17}$$

3. Candidate hidden state 3.18: This operation creates a vector of new candidate values ($\tilde{h}_t$) that could be added to the hidden state. It uses a tanh activation function to ensure the values are between -1 and 1.

$$\tilde{h}_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot (r_t \odot h_{t-1}) + b_h) \tag{3.18}$$

4. New hidden state 3.19: The new hidden state (h_t) is computed by combining the previous hidden state (h_{t-1}) scaled by the update gate and the candidate hidden

state ($\tilde{h}_t$) scaled by one minus the update gate.

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{3.19}$$

5. Output 3.20: The output vector (y_t) is determined from the new hidden state and the bias vector (b_y).

$$y_t = W_{hy} \cdot h_t + b_y \tag{3.20}$$

The training process of the GRU network is similar to that of the LSTM network, involving (BPTT) to optimize the weight matrices and bias vectors to minimize the defined loss ($\mathcal{L}$) over a dedicated training dataset.

Implementation and results

The implementation and training procedure of the described GRU network is identical to the one used for the RNN-s and LSTM-s. The same reduced input set (12 inputs) was used. A grid search was performed over the hyperparameters to find the best combination of parameters that yield the best architecture. The training was done in a similar manner as with the RNN-s and LSTM-s. For the first batch of grid search the hyperparameter ranges are the as in table 3.1.

The results obtained after testing the created models on the same unseen test dataset. The code using for training and validating can be found at `AI_training/3_code/training_GRU.py`. The 2 best performing architectures can be seen in table 3.12. The results that can be found in table 3.13 show that the GRU models outperformed the RNN models, but were slightly worse than the LSTM models, indicating that the GRU architecture is better suited for capturing the temporal dependencies in the vehicle speed estimation task than RNN-s, but not as good as LSTM-s.

	Input ID	**Sequence Length**	**Hidden size**	**Number of layers**
1	2	TBD	TBD	TBD
2	2	TBD	TBD	TBD

Table 3.12: Results of the best performing GRU architecture on the test dataset

	RMSE [$\frac{km}{h}$]	**MAE [$\frac{km}{h}$]**	**AMaxE [$\frac{km}{h}$]**	**PWT 0.3 [%]**	**PWT 0.4 [%]**
1	TBD	TBD	TBD	TBD	TBD
2	TBD	TBD	TBD	TBD	TBD

Table 3.13: Performance metrics of the best performing GRU architectures on the test dataset

After evaluating these results another grid search was performed around the 2 best performing architectures to see if further improvements could be achieved by finetuning

the hyperparameters. The hyperparameter ranges for the finetuning can be seen in table 3.14.

Around 1st best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	
Number of layers	
Sequence length	
Around 2nd best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Hidden size (d_h)	
Number of layers	
Sequence length	

Table 3.14: Batch 2: Hyperparameter ranges for GRU architecture search

The best model obtained from the second batch of grid search was evaluated on the unseen test dataset. The results of this best model can be seen in table 3.15.

RMSE [$\frac{km}{h}$]	**MAE** [$\frac{km}{h}$]	**AMaxE** [$\frac{km}{h}$]	**PWT 0.3 [%]**	**PWT 0.4 [%]**
TBD	TBD	TBD	TBD	TBD

Table 3.15: Performance metrics of the best performing GRU architecture from the second batch on the test dataset

3.3 Temporal Convolutional Networks - TCN

Theoretical background

The Temporal Convolutional Network (TCN) is a type of convolutional neural network (CNN) that is specifically designed for handling sequential data. Unlike RNN-s and their variants, TCN-s utilize convolutional layers to capture temporal dependencies in the data. The key features of TCN-s include causal convolutions, dilated convolutions, and residual connections.

The TCN produces the output sequence that has the same length as the input sequence. This is due to using a 1D fully convolutional network (FCN) architecture, meaning that each hidden layer is the same length as the input sequence. This property is fulfilled by using zero-padding at the beginning of the input sequence.

Causal convolutions ensure that the output at time step y_t is only dependent on the inputs from time steps t and earlier. Dilated convolutions allow the network to have a larger receptive field without increasing the number of parameters significantly. This is

achieved by introducing gaps between the filter elements, enabling the network to capture long-term dependencies more effectively. The dilated causal convolutions, i.e the base of the TCN, can be seen on figure 3.9, where the dilation factor d determines the spacing between the filter elements and the kernel size k defines the size of the filter.

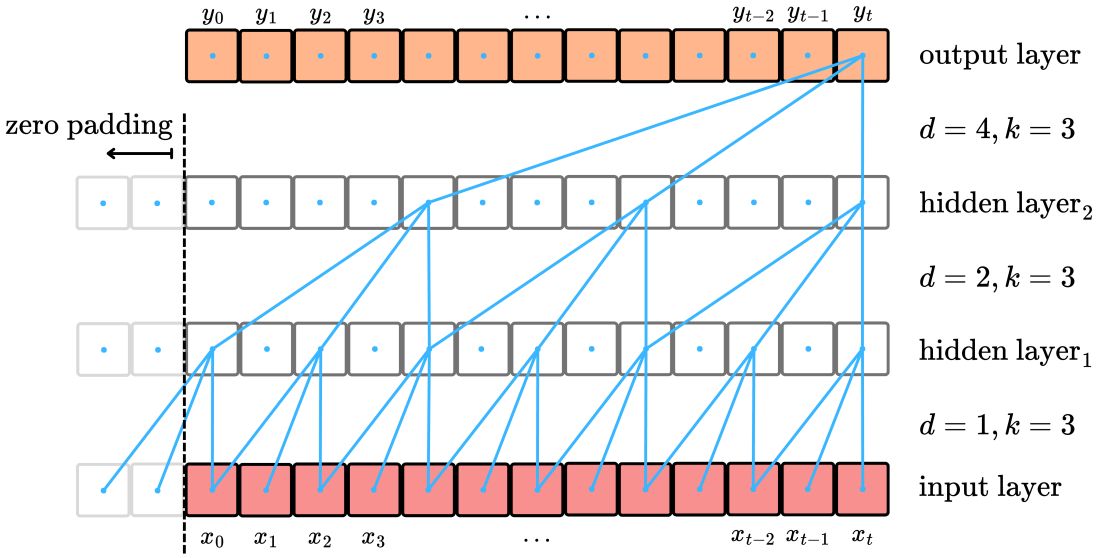


Figure 3.9: Dilated causal convolution

The dimensions of the vectors are the following:

- Input vector at time step t: $x_t \in \mathbb{R}^{d_x}$, where d_x is the channel size of the input vector. Channel size refers to the number of features in the input vector at each time step.
- Output vector at time step t: $y_t \in \mathbb{R}^{d_y}$, where d_y is the chanel size of the output vector.
- Sequence length: L, the total number of time steps in the input sequence.
- Input tensor: $X_t = [x_0, x_1, ..., x_t]^T \in \mathbb{R}^{L \times d_x}$
- Output tensor: $Y_t = [y_0, y_1, ..., y_t]^T \in \mathbb{R}^{L \times d_y}$

Based on the dilation and the kernel size between two layers, the equation for determining the output can the formulated as (if there are no hidden layers):

$$y_t = \sum_{i=0}^{k-1} w_i \cdot x_{t-d \cdot i} + b, \tag{3.21}$$

where y_t is the output vector at time step t, w_i are the weights of the filter, b is the bias term, k is the kernel size, and d is the dilation factor between the two layers.

The full equation considering the input and output tensors between two layers can be formulated as:

$$Y_{j,t} = \sum_{i=1}^{d_x} \sum_{m=0}^{k-1} W_{j,i,m} \cdot X_{i,t-d \cdot m} + b_j, \tag{3.22}$$

where $Y_{j,t}$ is the jth vector element of the output tensor at time step t, $W_{i,j,m}$ contains the weights of the filter connecting input channel i to output channel j at position m (in the kernel), and b_j is the bias term for output channel j. Iterating this equation all the vector elements of tensor Y can be determined based on the elements of the input tensor X.

This can be generalized for two arbitrary layers that are connected through a 1D dilated causal convolution as:

$$H_{j,t}^{(l)} = \sum_{i=1}^{d_h^{(l-1)}} \sum_{m=0}^{k-1} W_{j,i,m}^{(l)} \cdot H_{i,t-d^{(l)} \cdot m}^{(l-1)} + b_j^{(l)} = \text{Conv1D}(H^{(l-1)}), \tag{3.23}$$

where $H_{j,t}^{(l)}$ is the jth vector element in time step t of the output tensor of layer l, $d_h^{(l-1)}$ is the number of channels in the tensor of layer $l-1$, $W_{j,i,m}^{(l)}$ contains the weights of the filter connecting input channel i to output channel j at position m and $b_j^{(l)}$ is the jth bias for layer l.

After each convolutional layer, a non-linear activation function is applied (e.g., ReLU or tanh) to introduce non-linearity into the model. Moreover, a dropout and a normalization layer is often added after the activation function to prevent overfitting by randomly setting a fraction of the input units to zero during training. Finally a residual connection is added to the output of the convolutional block. Here it is inportant to ensure that the input and the output have the same dimensions to ensure the element-wise addition. These steps can be summarized as:

$$H^{(l)} = \text{Dropout}(\text{Activation}(\text{Conv1D}(H^{(l-1)}))) + \text{Conv1x1}(H^{(l-1)}) \tag{3.24}$$

These steps build up a single TCN residual block that can be seen on figure 3.10. By stacking multiple such blocks on top of each other, a deep TCN architecture can be created that is capable of capturing complex temporal dependencies in the input data.

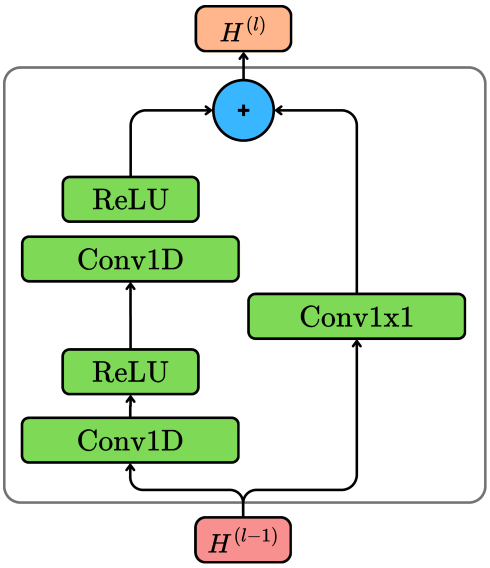


Figure 3.10: TCN residual block with two dilated causal convolutional layers

!! COULD TALK ABOUT THE RECEPTIVE FIELD HERE !!

During the training process, the objective is the same as with the RNN models - to optimize the weight matrices and bias vectors to minimize the defined loss ($\mathcal{L}$) over a dedicated training dataset. The training is porformed just as with a FNN, using back-propagation to calculate the gradients of the loss with respect to the weights and biases, and updating them using an optimizer such as Adam or SGD.

Implementation and results

The implementation and training procedure of the TCN network is similar to the on used in the RNN based models. The same reduced input set was used which consists of 12 inputs. A grid search was performed over the parameter space. The parameters that were varied during the grid search are the following: kernel size (k), dilation schedule between the layers ($d_1, d_2, ...d_n$), channels per layer (d_h), number of residual blocks (L) sequence length (L_{seq}) and convolutions per residual block (B). The value ranges for the first batch of the grid search can be seen in table 3.16.

Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Kernel size (k)	3, 4, 5
Dilation schedule (d_{sched})	[1, 2, 4, 8], [1, 2, 4, 8, 16], [1, 2, 4, 8, 16, 32]
Channels per layer (d_h)	64
Number of residual blocks (L)	4, 5, 6
Convolutions per residual block (B)	2
Sequence length (L_{seq})	50, 100, 150, 200

Table 3.16: Batch 1: Hyperparameter ranges for TCN architecture search

After the training and evaluation of the models from the first batch of grid search, the 2 best performing architecture was selected based on the same principle as before. The results of these best models can be seen in table 3.18. The code using for training and validating can be found at `AI_training/3_code/training_TCN.py`.

	k	d_{sched}	d_h	L	L_{seq}
1	TBD	TBD	TBD	TBD	TBD
2	TBD	TBD	TBD	TBD	TBD

Table 3.17: Results of the best performing TCN architecture on the test dataset

RMSE $[\frac{km}{h}]$	**MAE** $[\frac{km}{h}]$	**AMaxE** $[\frac{km}{h}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
TBD	TBD	TBD	TBD	TBD
TBD	TBD	TBD	TBD	TBD

Table 3.18: Performance metrics of the best performing TCN architectures from the first batch on the test dataset

After evaluating these results another grid search was performed around the 2 best performing architectures to see if further improvements could be achieved by finetuning the hyperparameters. The hyperparameter ranges for the finetuning can be seen in table 3.19.

Around 1st best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Kernel size (k)	
Dilation schedule (d_{sched})	
Channels per layer (d_h)	
Number of residual blocks (L)	
Sequence length (L_{seq})	
Around 2nd best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Kernel size (k)	
Dilation schedule (d_{sched})	
Channels per layer (d_h)	
Number of residual blocks (L)	
Sequence length (L_{seq})	

Table 3.19: Batch 2: Hyperparameter ranges for TCN architecture search

The best model obtained from the second batch of grid search was evaluated on the unseen test dataset. The results of this best model can be seen in table 3.20.

RMSE [$\frac{km}{h}$]	MAE [$\frac{km}{h}$]	AMaxE [$\frac{km}{h}$]	PWT 0.3 [%]	PWT 0.4 [%]
TBD	TBD	TBD	TBD	TBD

Table 3.20: Performance metrics of the best performing TCN architecture from the second batch on the test dataset

!! Here I could write that the TCN models performed much better than the rnn based ones !!

3.4 Transformer

Theoretical background

The Transformer architecture is a deep learning model that has revolutionized the field of natural language processing (NLP) and has also found applications in other sequential data tasks. Unlike traditional RNN-s and TCN-s where the models rely on recurrence or convolution, the transformer relies entirely on self-attention mechanisms to capture dependencies in the input data, allowing for parallel processing of sequences and improved handling of long-range dependencies. An other important aspect of the transformer architecture is that it does not take the sequence data in a step-by-step manner, but rather processes the entire sequence at once, making it different from the previously discussed models.

The general architecture of the transformer model can be divided into two main components: the encoder and the decoder. The encoder is responsible for processing the input sequence and generating a set of continuous representations, while the decoder takes these representations and generates the output sequence. However, for tasks such as vehicle speed estimation, only the encoder part is enough, as there is no need to generate a sequence output.

The steps of the transformer encoder can be seen on 3.11 and summarized as follows:

1. The input sequence is

2. Input embedding: The input sequence $X \in \mathbb{R}^{L \times d_x}$ at time step t, where L is the sequence length and d_x is the size of the input vector, is first converted into a continuous representation using an embedding layer. This layer maps each input to a high-dimensional vector space d_m.

$$E_t = \text{Embedding}(X_t) = W_e \cdot X_t + b_e, \tag{3.25}$$

where $E_t \in \mathbb{R}^{L \times d_m}$ is the embedded input vector at time step t, $W_e \in \mathbb{R}^{d_m}$ is the embedding weight matrix, and $b_e \in \mathbb{R}^{d_m}$ is the bias term.

3. Positional encoding: Since the transformer architecture does not rely on the order of the sequence, positional encodings are added to the embedded input vectors to provide information about the position of each element in the sequence. This can be done using sinusoidal functions or learned positional embeddings. The original transformer paper [**Vaswani2017AttentionIA**] proposes the following sinusoidal positional encoding:

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_m}}\right), \tag{3.26}$$

$$\mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_m}}\right), \tag{3.27}$$

where pos is the position in the sequence and i is the dimension index. After the positional encodings are calculated, they are added to the embedded input vectors:

$$X'_t = E_t + PE, \tag{3.28}$$

where X'_t is the input embedded vector with added positional encodings.

4. Multi-head self-attention: This mechanism allows the model to focus on different parts of the input sequence simultaneously. The input is transformed into three matrices: queries (Q), keys (K), and values (V) using learned weight matrices (W_Q, W_K and W_V). The attention scores are calculated using the following equation:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{3.29}$$

where d_k is the dimension of the key vectors. The term multi-head attention implies that the process is repeated multiple times (heads) with different learned weight matrices, allowing the model to capture various aspects of the input sequence. The outputs of each head are then concatenated and transformed using another learned weight matrix (W_O):

$$\mathrm{head}_i = \mathrm{Attention}(X'_t W_{Q_i}, X'_t W_{K_i}, X'_t W_{V_i}), \tag{3.30}$$

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \mathrm{head}_2, ..., \mathrm{head}_h)W_O, \tag{3.31}$$

where h is the number of attention heads.

5. Add and Normalization: After the multi-head self-attention layer, a residual connection is added that is followed by layer normalization. This helps in stabilizing the training process and allows for better gradient flow.

$$X_{attn} = \mathrm{LayerNorm}(X'_t + \mathrm{MultiHead}(X'_t)), \tag{3.32}$$

where $X_{attn} \in \mathbb{R}^{L \times d_m}$ is the output of the multi-head self-attention layer at time step t.

6. Feed-forward neural network: After the multi-head self-attention layer, a position-wise feed-forward neural network (FFN) is applied to each position in the sequence independently. This FFN consists of two linear transformations with an activation function (eg. ReLU):

$$\text{FFN}(X_{attn}) = W_2 \text{act}(W_1 X_{attn} + b_1) + b_2, \tag{3.33}$$

where $W_1 \in \mathbb{R}^{d_m \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d_m}$ are the weight matrices, and $b_1 \in \mathbb{R}^{d_{ff}}$, $b_2 \in \mathbb{R}^{d_m}$ are the bias terms. d_{ff} is the dimension of the feed-forward layer.

7. Add and Normalization: Similar to the previous step, a residual connection is added followed by layer normalization after the feed-forward neural network.

$$Y_t = \text{LayerNorm}(X_{attn} + \text{FFN}(X_{attn})), \tag{3.34}$$

where $Y_t \in \mathbb{R}^{L \times d_m}$ is the output of the transformer encoder at time step t.

8. Regression head: Finally, a regression head is added on top of the transformer encoder to map the output representations to the desired output size ie. the vehicle speed. This can be done using a simple feed-forward layer:

$$\hat{y}_t = W_{out} \cdot Y_t + b_{out}, \tag{3.35}$$

where $\hat{y}_t \in \mathbb{R}^{L \times d_y}$ is the predicted output vector at time step t, $W_{out} \in \mathbb{R}^{d_m \times d_y}$ is the output weight matrix, and $b_{out} \in \mathbb{R}^{d_y}$ is the bias term. From here the last element of the output vector can be taken as the estimated vehicle speed at the current time step.

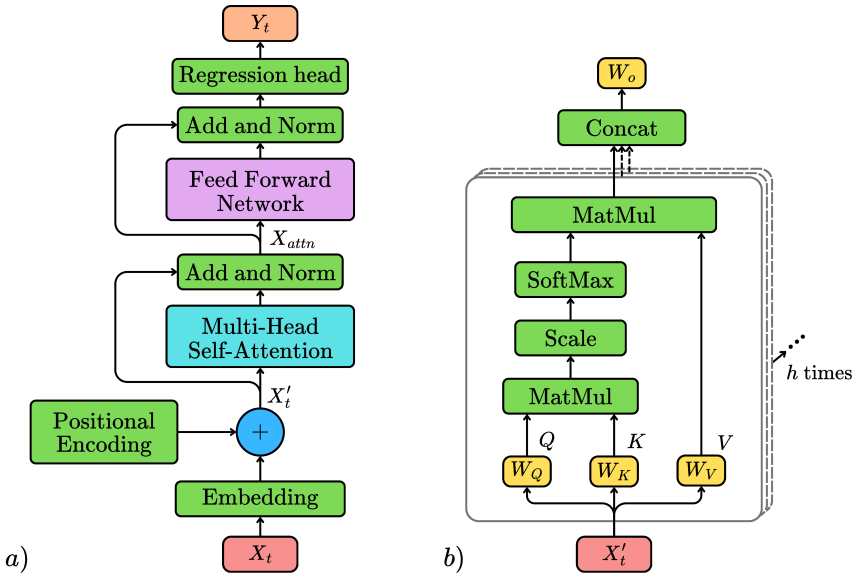


Figure 3.11: a) Transfomer encoder block with regression head b) Multi-head self-attention mechanism

Implementation and results

The implementation and training procedure of the transformer encoder is similar to the one used in the previous models. The same reduced input set was used which consists of 12 inputs. A grid search was performed over the parameter space. The parameters that were varied during the grid search are the following: model dimension (d_m), feed-forward dimension (d_{ff}), number of layers (L), number of attention heads (n_{head}) and sequence length (L_{seq}). The value ranges for the first batch of the grid search can be seen in table 3.21.

Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Model dimension (d_m)	32, 64
Feed-forward dimension (d_{ff})	64, 128, 96
Number of layers (L)	2, 3
Number of attention heads (n_{head})	2, 4
Sequence length (L_{seq})	50, 100, 150, 200

Table 3.21: Batch 1: Hyperparameter ranges for transformer architecture search

The results of the best 2 models obtained from the first batch (3.22) of grid search can be seen in table 3.23. The code using for training and validating can be found at `AI_training/3_code/training_transformer.py`.

	d_m	d_{ff}	L	n_{head}	L_{seq}
1	TBD	TBD	TBD	TBD	TBD
2	TBD	TBD	TBD	TBD	TBD

Table 3.22: Results of the best performing transformer architecture on the test dataset

RMSE $[\frac{km}{h}]$	**MAE** $[\frac{km}{h}]$	**AMaxE** $[\frac{km}{h}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
TBD	TBD	TBD	TBD	TBD
TBD	TBD	TBD	TBD	TBD

Table 3.23: Performance metrics of the best performing transformer architectures from the first batch on the test dataset

After evaluating these results another grid search was performed around the 2 best performing architectures to see if further improvements could be achieved by finetuning the hyperparameters. The hyperparameter ranges for the finetuning can be seen in table 3.24.

Around 1st best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Model dimension (d_m)	
Feed-forward dimension (d_{ff})	
Number of layers (L)	
Number of attention heads (n_{head})	
Sequence length (L_{seq})	
Around 2nd best	
Hyperparameters	**Possible values**
Number of inputs (d_x)	12
Model dimension (d_m)	
Feed-forward dimension (d_{ff})	
Number of layers (L)	
Number of attention heads (n_{head})	
Sequence length (L_{seq})	

Table 3.24: Batch 2: Hyperparameter ranges for transformer architecture search

The best model obtained from the second batch of grid search was evaluated on the unseen test dataset. The results of this best model can be seen in table 3.25.

RMSE $[\frac{km}{h}]$	**MAE** $[\frac{km}{h}]$	**AMaxE** $[\frac{km}{h}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
TBD	TBD	TBD	TBD	TBD

Table 3.25: Performance metrics of the best performing transformer architecture from the second batch on the test dataset

3.5 Summary and comparison of the sequential models

In this section the results of the best performing sequential models are summarized and compared against each other. The results of the best RNN, LSTM, GRU, TCN and transformer models can be seen in table ??.

Chapter 4

AI aided estimation

In the previous chapter the direct estimation of the vehicle speed with different AI models was discussed. In this chapter the focus will be shifted towards combining conventional estimation methods with AI algorithms to improve the overall performance of the estimator. First a simple wheel speed based vehicle speed estimation method will be presented. Then a model based estimator will be discussed that utilizes the linear tire model to estimate the vehicle speed from the tire forces. Furthermore an Extended Kalman Filter (EKF) based estimator will be developed that fuses the previously derived model based estimator as the state transition function and the wheel speed based estimator as the measurement function. Finally, the best performing AI models from the previous chapter will be combined with the EKF based estimator to improve its performance.

An important notice is that all the newly presented estimation methods are based on the simplification that the vehicle is a 2D rigid body moving on a flat surface. Therefore the vertical dynamics of the vehicle are neglected and the roll and pitch anlges are considered to be zero.

4.1 Best wheel based speed estimation

Governing equations

The best wheel based speed estimation method is a simple yet effective approach to estimate the vehicle speed using the rotational speeds of the individual wheels. The main idea behind it is to calculate the vehicle speed from the rotational speeds of the wheels, transform these speeds into the Center of Gravity (CoG) of the vehicle and then take the best estimate from the four wheels.

The best estimate is determined based on the vehicle's longitudinal acceleration. If the vehicle is accelerating ($a_x > 0$), the wheel with the minimum angular velocity is chosen as the best estimate, as it is less likely to be affected by wheel slip. WIth the same approach, if the vehicle is decelerating ($a_x < 0$), the wheel with the maximum angular velocity is

selected, as it is less likely to be affected by braking slip.

The sensor measurements needed for this estimation method are the four wheel speeds (ω^{FL}, ω^{FR}, ω^{RL}, ω^{RR}), the longitudinal acceleration (a_x) of the vehicle, the yaw rate (ω_z) of the vehicle and the road wheel angles (δ^{FL}, δ^{FR}, δ^{RL}, δ^{RR}). These are all available from the vehicle's Inertial Measurement Unit (IMU) and wheel speed sensors. Furthermore the vehicle's geometric parameters such as the effective wheel radius (R) and the positional vectors pointing from the COG to the wheels (r^{FL}, r^{FR}, r^{RL}, r^{RR}) are also required.

The calculation steps of the described estimator are the following. Calculation of the velocity at each wheel (v_x^i) from the wheel speeds (ω^i)

$$v_{w,x}^i = w_i \cdot R, \tag{4.1}$$

where $i \in \{FR, FL, RR, RL\}$. This way the longitudinal velocity at each wheel is obtained,

$$v_w^i = [v_{w,x}^i, 0, 0]^T. \tag{4.2}$$

Here it needs to mentioned that this is just an approximation, as the lateral velocity component at the wheel is neglected. The next step is to express the determined wheel velocity vectors in the frame that is aligned with the frame of the body but is centered at the wheels. This is done by applying a rotation matrix (T_{δ_i}) based on the road wheel angles (δ^i) of the respective wheels. The rotation matrix is the can be written as

$$T_{\delta^i} = \begin{bmatrix} \cos(\delta^i) & -\sin(\delta^i) & 0 \\ \sin(\delta^i) & \cos(\delta^i) & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{4.3}$$

After rotating the wheel velocities to the appropriate frame, the velocity at the CoG of the vehicle can be calculated by using the yaw rate ($\omega = [0, 0, \omega_z]^T$) of the vehicle and the positional vectors pointing from the COG to the wheels (r_i). The velocity at the CoG can be calculated as

$$v_{cog}^i = T_{\delta^i} \cdot v_w^i + \omega \times r^i, \tag{4.4}$$

where v_{cog}^i are the velocity vectors at the CoG calculated from the four different wheels. After calculating the CoG velocities from each wheel, the best estimate can be selected based on the longitudinal acceleration (a_x) of the vehicle. The final vehicle speed estimate (v_{est}) can be written as

$$v_{est} = \begin{cases} \min(v_{cog}^{FR}, v_{cog}^{FL}, v_{cog}^{RR}, v_{cog}^{RL}) & \text{if } a_x \geq 0 \\ \max(v_{cog}^{FR}, v_{cog}^{FL}, v_{cog}^{RR}, v_{cog}^{RL}) & \text{if } a_x < 0 \end{cases}. \tag{4.5}$$

Implementation and results

The best wheel based speed estimator was implemented in MATLAB Simulink envionment. The function that realizes the described calculation can be found at `Conventional_estimators/Best_wheel_based_speed_estimator.m`. The estimator was tested on the same test dataset as the AI based direct estimators from the previous chapter. The results of the evaluation can be seen in table 4.1.

The parameters of the vehicle that were used for the estimation are the following: effective wheel radius R, positional vectors from COG to wheels: r^{FL}, r^{FR}, r^{RL}, r^{RR}. The values of the parameters can be found in table 4.2.

RMSE $[\frac{km}{h}]$	**MAE** $[\frac{km}{h}]$	**AMaxE** $[\frac{km}{h}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
TBD	TBD	TBD	TBD	TBD

Table 4.1: Performance metrics of the best wheel based speed estimator on the test dataset

4.2 Model based speed estimation

Governing equations

In this section the intention is to create a function that can determine the vehicle speeds time derivatives based on the available sensore measurements and the vehicle parameters. The developed model based estimator will be later used as the state transition function in the EKF based estimator.

$$\dot{x} = f(x, u), \tag{4.6}$$

where $x = [v_x, v_y, \omega_z]^T$ is the chosen state vector containing the longitudinal speed, the lateral speed and the yaw rate of the vehicle. The input vector is defined as:

$$u = [\omega^i, \delta^i, m, I_z, c^j_{lon}, c^j_{lat}, R, r^i, dt, \epsilon]^T, \tag{4.7}$$

where ω^i are the four wheel speeds, δ^i are the four road wheel angles, m is the mass of the vehicle, I_z is the yaw moment of inertia of the vehicle, c^j_{lon} and c^j_{lat} are the longitudinal and lateral tire stiffness coefficients for the front and rear wheels, R is the effective wheel radius, r^i are the positional vectors pointing from the COG to the wheels, dt is the sampling time and ϵ is a small constatn that will be used at the longitudinal slip calculation. The indices $i \in \{FR, FL, RR, RL\}$ and $j \in \{F, R\}$.

The first step of the state transition function is to determine the velocity vectors at each wheel (v^i_w). For velocity at each wheel needs to be calulated. This can be done the

same way as in the previous section:

$$v_w^i = T_{\delta^i} \cdot (v_{cog} - \omega \times r^i), \tag{4.8}$$

where $v_{cog} = [v_x, v_y, 0]^T$, $\omega = [0, 0, \omega_z]^T$, T_{δ^i} is the rotation matrix based on the road wheel angles (δ^i) of the respective wheels (same as in equation 4.3). The next step is to calculate the longitudinal slip ratios (κ^i) and lateral side slip angles (α^i) at each wheel. The slip ratio can be calculated as

$$\kappa^i = \frac{\omega^i \cdot R - v_{w,x}^i}{\max(|v_{w,x}^i|, \epsilon)}, \tag{4.9}$$

where ϵ is a small constant to avoid division by zero. The slip angle at each wheel can be determined as

$$\alpha^i = \arctan\left(\frac{-v_{w,y}^i}{\max(|v_{w,x}^i|, \epsilon)}\right). \tag{4.10}$$

From here the tire forces at each wheel will be determined using the linear tire model. The longitudinal tire forces (F_x^i) can be calculated as

$$F_x^i = c_{lon}^j \cdot \kappa^i, \tag{4.11}$$

where $j \in \{F, R\}$ depending on whether the wheel is a front or rear wheel. The lateral tire forces (F_y^i) can be determined as

$$F_y^i = c_{lat}^j \cdot \alpha^i. \tag{4.12}$$

With this the tire force vectors at each wheel can be expressed as

$$F_{tire}^i = [F_x^i, F_y^i, 0]^T. \tag{4.13}$$

After determining the tire forces at each wheel, the total force acting on the vehicle can be calculated by transforming the tire forces to the vehicle's body frame and summing them up:

$$F_{total} = \sum_i T_{-\delta^i}^T \cdot F_{tire}^i, \tag{4.14}$$

From the transformed tire forces at the wheels the total moment acting on the vehicle around the vertical axis can be calculated as

$$M_z = \sum_i (r^i \times (T_{-\delta^i}^T \cdot F_{tire}^i))_z. \tag{4.15}$$

With the knowledge of the total force and moment acting on the vehicle, the time deriva-

tives of the state variables can be calculated as

$$\dot{v}_x = \frac{F_{total,x}}{m} + \omega_z \cdot v_y, \tag{4.16}$$

$$\dot{v}_y = \frac{F_{total,y}}{m} - \omega_z \cdot v_x, \tag{4.17}$$

$$\dot{\omega}_z = \frac{M_z}{I_z}. \tag{4.18}$$

In order to use the described state transition function in a discrete time domain, an integration step is needed to obtain the state variables at the next time step. This can be done with a forward Euler method. The integrations step can be written as

$$x[k] = x[k-1] + f(x[k-1], u[k-1]) \cdot dt. \tag{4.19}$$

Implementation and results

The model based speed estimator was also implemented in MATLAB Simulink envionment. The function that realizes the described calculation can be found at `Conventional_estimators/Model_based_speed_estimator.m`. The estimator was tested on the same test dataset and the results of the evaluation can be seen in table 4.3.

The model parameters that were used for the estimation can be seen in table 4.2.

Parameter	**Value**
Vehicle mass (m)	3020 $[kg]$
Yaw moment of inertia (I_z)	7145 $[kgm^2]$
Front longitudinal tire stiffness (c_{lon}^F)	180000 $[N]$
Rear longitudinal tire stiffness (c_{lon}^R)	200000 $[N]$
Front lateral tire stiffness (c_{lat}^F)	140000 $[N]$
Rear lateral tire stiffness (c_{lat}^R)	150000 $[N]$
Effective wheel radius (R)	0.3615 [m]
Positional vector to front left wheel (r^{FL})	$[1.675, 0.831, 0]^T$ [m]
Positional vector to front right wheel (r^{FR})	$[1.675, -0.831, 0]^T$ [m]
Positional vector to rear left wheel (r^{RL})	$[-1.540, 0.842, 0]^T$ [m]
Positional vector to rear right wheel (r^{RR})	$[-1.540, -0.842, 0]^T$ [m]
Small constant for slip calculation (ϵ)	1

Table 4.2: Model parameters used for the model based speed estimator

RMSE $[\frac{km}{h}]$	**MAE** $[\frac{km}{h}]$	**AMaxE** $[\frac{km}{h}]$	**PWT 0.3 [%]**	**PWT 0.4 [%]**
TBD	TBD	TBD	TBD	TBD

Table 4.3: Performance metrics of the model based speed estimator on the test dataset

4.3 Extended Kalman Filter based estimation

4.3.1 EKF theory

The Extended Kalman Filter (EKF) is an extension of the original Kalman Filter (KF) that is designed to handle non-linear systems. While the standard KF assumes that both the state transition and measurement functions are linear, the EKF allows for non-linear functions by linearizing them around the current estimate. However the non-linear state transition and measurement functions need to be differentiable. The two functions can be expressed as:

$$x[k] = f(x[k-1], u[k-1]) + w[k-1], \tag{4.20}$$

$$z[k] = h(x[k]) + v[k], \tag{4.21}$$

where $x[k]$ is the state vector at time step k, $u[k-1]$ is the input vector at time step $k-1$, $z[k]$ is the measurement vector at time step k, f is the non-linear state transition function, h is the non-linear measurement function, $w[k-1]$ is the process noise and $v[k]$ is the measurement noise. Both noise terms are assumed to be zero-mean Gaussian with known covariance matrices Q and R respectively.

The EKF algorithm consists of two main steps: prediction and update. In the prediction step, the state predicted state estimate ($\hat{x}_{k|k-1}$) and the error covariance ($P_{k|k-1}$) are calculated using the non-linear state transition function:

$$\hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1}, u_{k-1}), \tag{4.22}$$

$$P_{k|k-1} = F_{k-1} P_{k-1|k-1} F_{k-1}^T + Q, \tag{4.23}$$

where F_{k-1} is the Jacobian of the state transition function evaluated at the previous state estimate:

$$F_{k-1} = \left.\frac{\partial f}{\partial x}\right|_{x=\hat{x}_{k-1|k-1}, u=u_{k-1}}. \tag{4.24}$$

In the update step, the Kalman gain (K_k), the updated state estimate ($\hat{x}_{k|k}$) and the updated error covariance ($P_{k|k}$) are calculated using the non-linear measurement function:

$$K_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R)^{-1}, \tag{4.25}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - h(\hat{x}_{k|k-1})), \tag{4.26}$$

$$P_{k|k} = (I - K_k H_k) P_{k|k-1}, \tag{4.27}$$

where H_k is the Jacobian of the measurement function evaluated at the predicted state

estimate:

$$H_k = \left.\frac{\partial h}{\partial x}\right|_{x=\hat{x}_{k|k-1}} . \tag{4.28}$$

4.3.2 EKF implementation

As mentioned in the prevous section, there are two main components that need to be defined in order to implement the EKF based estimator: the state transition function and the measurement function. The state transition function will be the previously developed model in section 4.2, that calculates the vehicle speed derivatives based on the vehicle model and the sensor measurements. The measurement function will be first the best wheel based speed estimator presented in section 4.1, that provides an estimate of the vehicle speed based on the wheel speeds. Later the best performing AI based direct vehicle speed estimator from section 3.5 will be used as the measurement function.

EKF with best wheel based speed estimation

EKF with AI based speed estimation

4.4 Real-time application

Summary

!! THIS IS AN AI GENERATED SUMMARY. PLEASE REVIEW AND EDIT AS NEEDED. !!

This thesis focused on the development of vehicle speed estimation algorithms based on artificial intelligence methods. The main objective was to develop a direct vehicle speed estimator using recurrent neural networks (RNNs) and compare its performance to a conventional wheel based speed estimator enhanced with AI based components. The direct speed estimator was developed using a dataset recorded from a test vehicle equipped with various sensors. The dataset included measurements such as wheel speeds, accelerations, yaw rate, and the ground truth vehicle speed obtained from a high-precision GPS system. The RNN architecture was chosen due to its ability to capture temporal dependencies in sequential data, making it suitable for vehicle speed estimation tasks. The development process involved data preprocessing, feature selection, model training, and evaluation. Various RNN architectures were explored, including Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks. Hyperparameter tuning was performed to optimize the model's performance. The direct speed estimator was evaluated using metrics such as root mean squared error (RMSE) and percentage within threshold (PWT). The results demonstrated that the RNN-based estimator achieved high accuracy in estimating vehicle speed, outperforming traditional methods in certain scenarios.

In addition to the direct speed estimator, a conventional wheel based speed estimator was developed and enhanced with AI based components. This estimator utilized wheel speed measurements and incorporated machine learning techniques to improve its accuracy. The performance of the conventional estimator was compared to the RNN-based direct estimator using the same evaluation metrics. The comparison revealed that while the conventional estimator performed well under normal driving conditions, the RNN-based estimator exhibited superior performance in scenarios involving rapid acceleration or deceleration, as well as varying road conditions.

Overall, this thesis demonstrated the effectiveness of RNNs in vehicle speed estimation tasks and highlighted the potential of AI based methods to enhance traditional estimation techniques. The developed direct speed estimator showed promising results and could be further improved with additional data and advanced architectures. Future work could explore the integration of other sensor modalities, such as camera or LiDAR data, to

further enhance the accuracy and robustness of vehicle speed estimation algorithms.

List of Figures

List of Tables

Appendix A

Appendix Title

At the end of the thesis, appendices can be used to include code snippets, catalog pages, large images, etc. This is referenced as 1. appendix in the text. It is recommended to use an online version control system (e.g., GitHub) to publicly upload the project and provide the link in the thesis.

Appendix B

Appendix Title

Model ID	Type	Input ID	Hidden size	Sequence length	Layers
153.1	RNN	2	50	64	1
153.2	RNN	1	50	64	1
154.1	RNN	2	100	64	1
154.2	RNN	1	100	64	1
155.1	RNN	2	150	64	1
155.2	RNN	1	150	64	1
156.1	RNN	2	200	64	1
156.2	RNN	1	200	64	1
157.1	RNN	2	50	96	1
157.2	RNN	1	50	96	1
158.1	RNN	2	100	96	1
158.2	RNN	1	100	96	1
159.1	RNN	2	150	96	1
159.2	RNN	1	150	96	1
160.1	RNN	2	200	96	1
160.2	RNN	1	200	96	1
161.1	RNN	2	50	128	1
161.2	RNN	1	50	128	1
162.1	RNN	2	100	128	1
162.2	RNN	1	100	128	1
163.1	RNN	2	150	128	1
163.2	RNN	1	150	128	1
164.1	RNN	2	200	128	1
164.2	RNN	1	200	128	1
165.1	RNN	2	50	192	1
165.2	RNN	1	50	192	1
166.1	RNN	2	100	192	1
166.2	RNN	1	100	192	1
167.1	RNN	2	150	192	1
167.2	RNN	1	150	192	1
168.1	RNN	2	200	192	1
168.2	RNN	1	200	192	1
169.1	RNN	2	50	64	2
169.2	RNN	1	50	64	2
170.1	RNN	2	100	64	2
170.2	RNN	1	100	64	2
171.1	RNN	2	150	64	2
171.2	RNN	1	150	64	2
172.1	RNN	2	200	64	2
172.2	RNN	1	200	64	2
173.1	RNN	2	50	96	2
173.2	RNN	1	50	96	2
174.1	RNN	2	100	96	2
174.2	RNN	1	100	96	2
175.1	RNN	2	150	96	2
175.2	RNN	1	150	96	2
176.1	RNN	2	200	96	2
176.2	RNN	1	200	96	2
177.1	RNN	2	50	128	2
177.2	RNN	1	50	128	2
178.1	RNN	2	100	128	2